Generative AI

(updated on 1 Nov 2026)

Professor Peerapon Vateekul, Ph.D.
peerapon.v@chula.ac.th

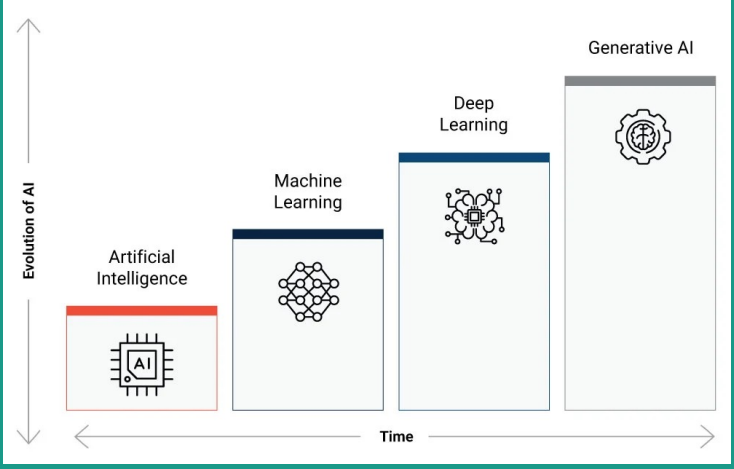
Department of Computer Engineering,
Faculty of Engineering, Chulalongkorn University

Outline

1. Generative AI & Modern LLMs
2. Prompt Engineering
3. Retrieval-Augmented Generation (RAG)
4. Fine-Tuning (briefly)
5. Tool Use & Agentic AI
6. Context Engineering, Evaluation & Safety, Harness Engineering
7. LLM Tools

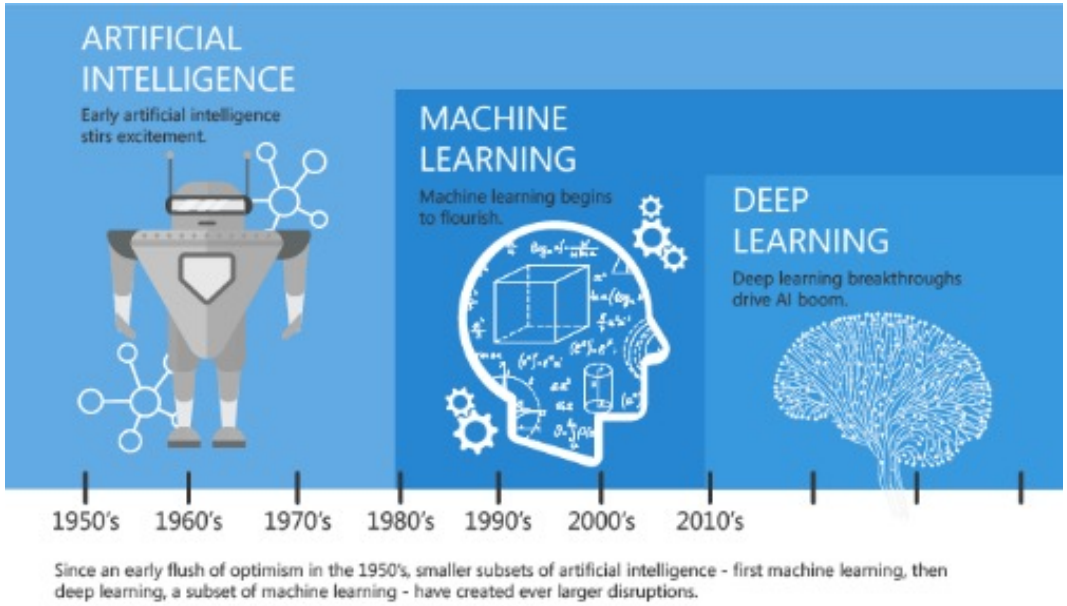
1) Generative AI & Modern LLMs

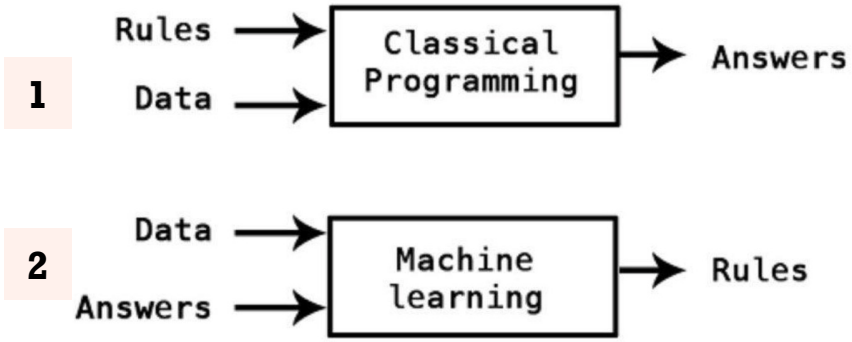
Credit to Asst. Prof.Ekapol's slide

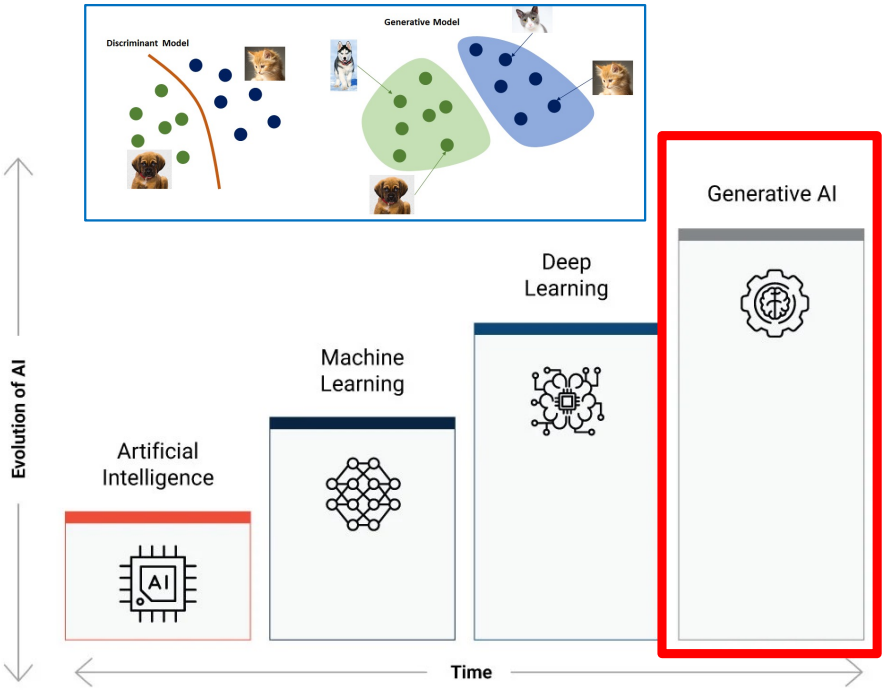


AI = Automation

- 1) Rule-based AI
- 2) Machine Learning (ML)





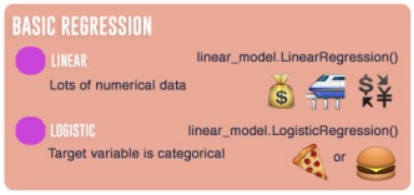


https://mc.ai/machine-learning-basics-artificial-intelligence-machine-learning-and-deep-learning/

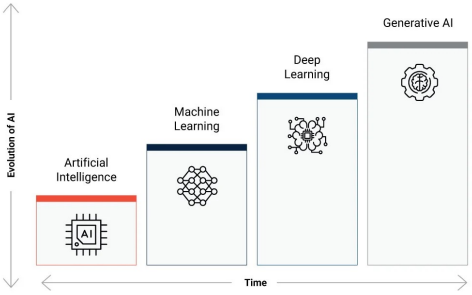
Past to Present

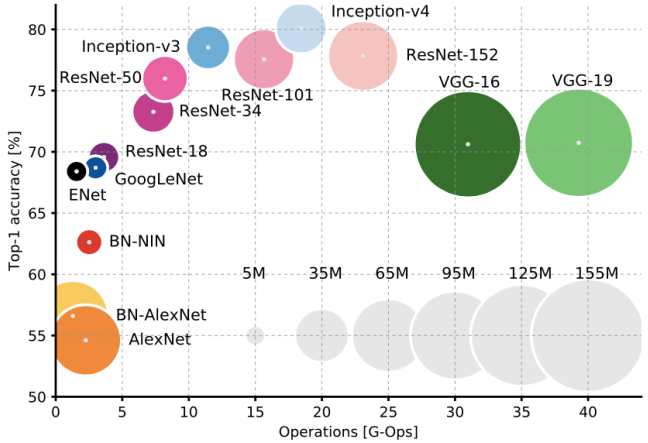
- Regression (few)
- NN (thoudsands)
- DL (millions)
- Gen AI (billions)

Spend = 500 + 10*Age + 20*Income1K

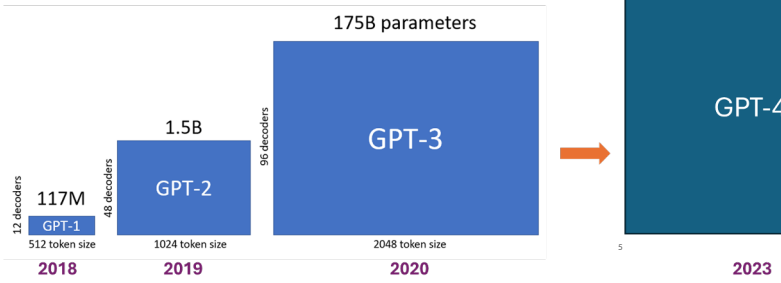


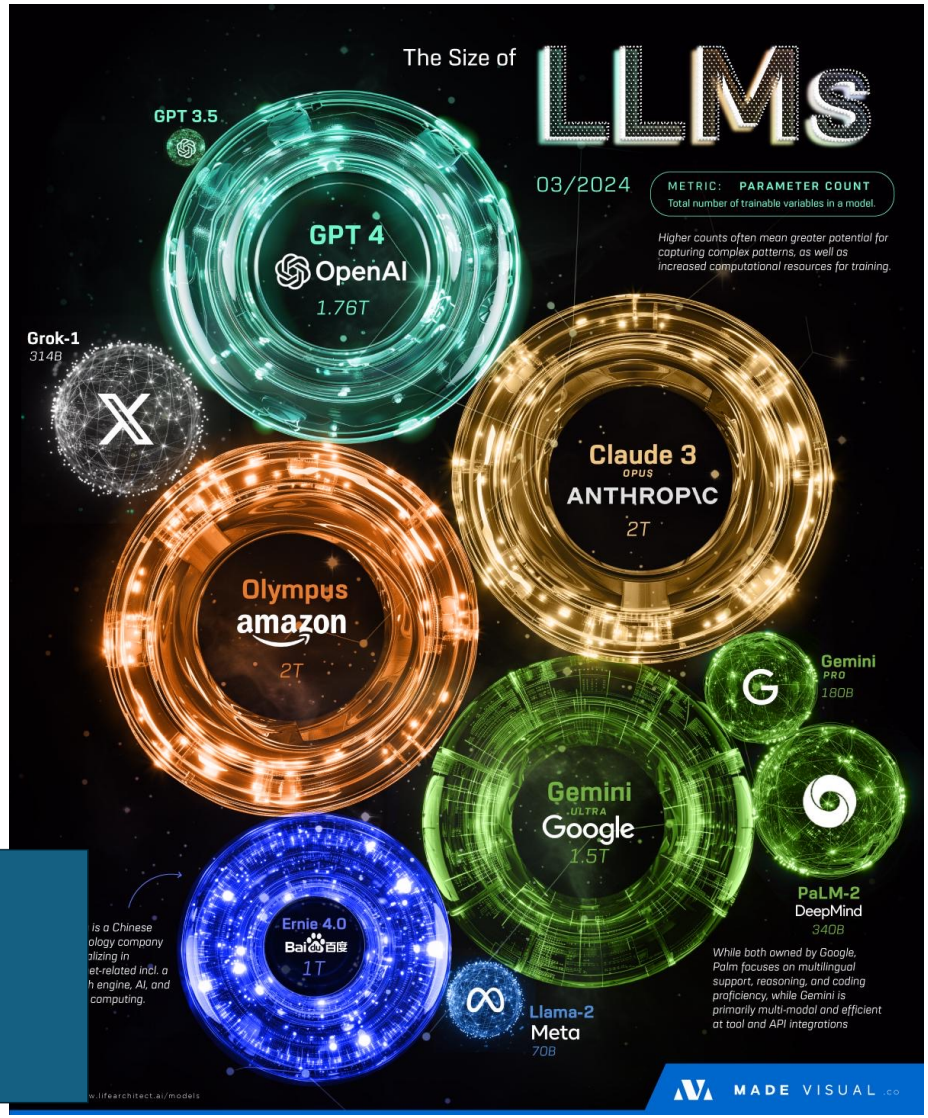






How exactly "Large" it is





Predictive AI vs. Generative AI

Predictive AI

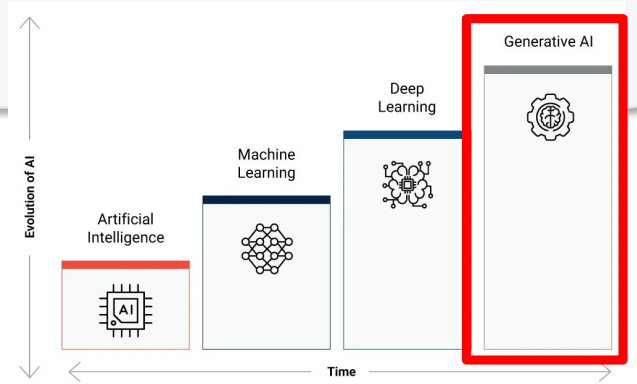
Input → Model → Prediction

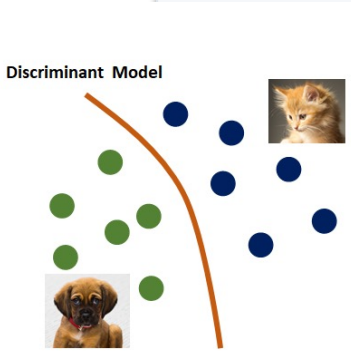
- Classification
- Regression / forecasting
- Detection / segmentation

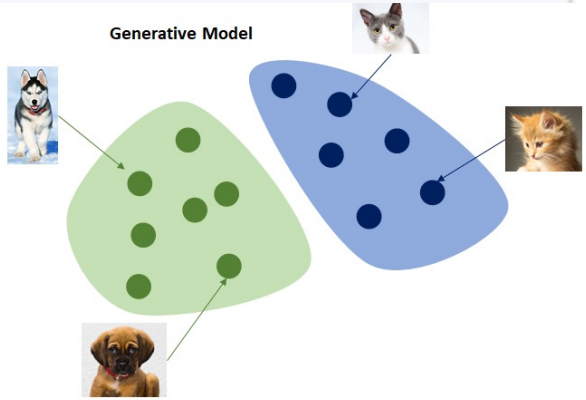
Generative AI

Prompt → Foundation Model → New Content

- Text & code
- Images / audio / video
- Structured outputs







ChatGPT

- ChatGPT was launched by OpenAI on 30 Nov 2022.
- ChatGPT is a large language model (LLM) for conversational AI applications.
- Generates human-like text and performs NLP tasks.
- Scalable and flexible for various use cases.
- ChatGPT didn't enclose the details.

OpenAI



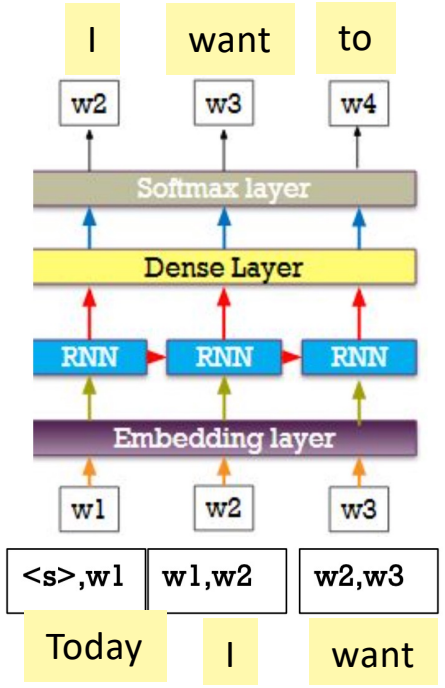
Language Model (LM)

- It is the model that aims to predict next word based on the given previous words.
- So, the model can understand grammar & context.

วันนี้เราอยากจะกินข้าวมัน____

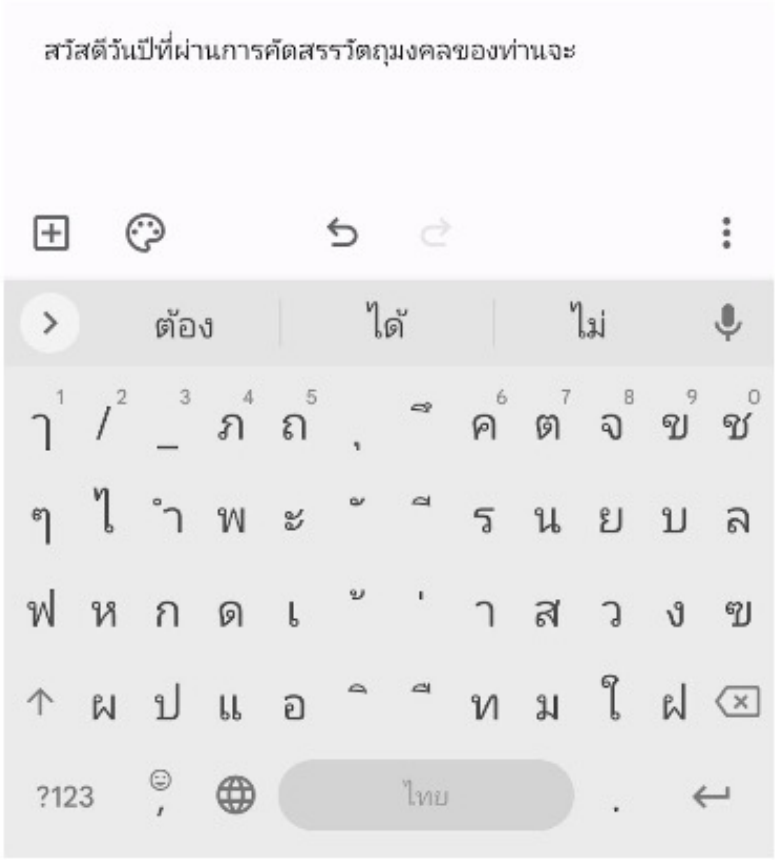
Today I want to eat chick ____ .

วัน นี้ เรา อยาก จะ กิน ข้าว มัน



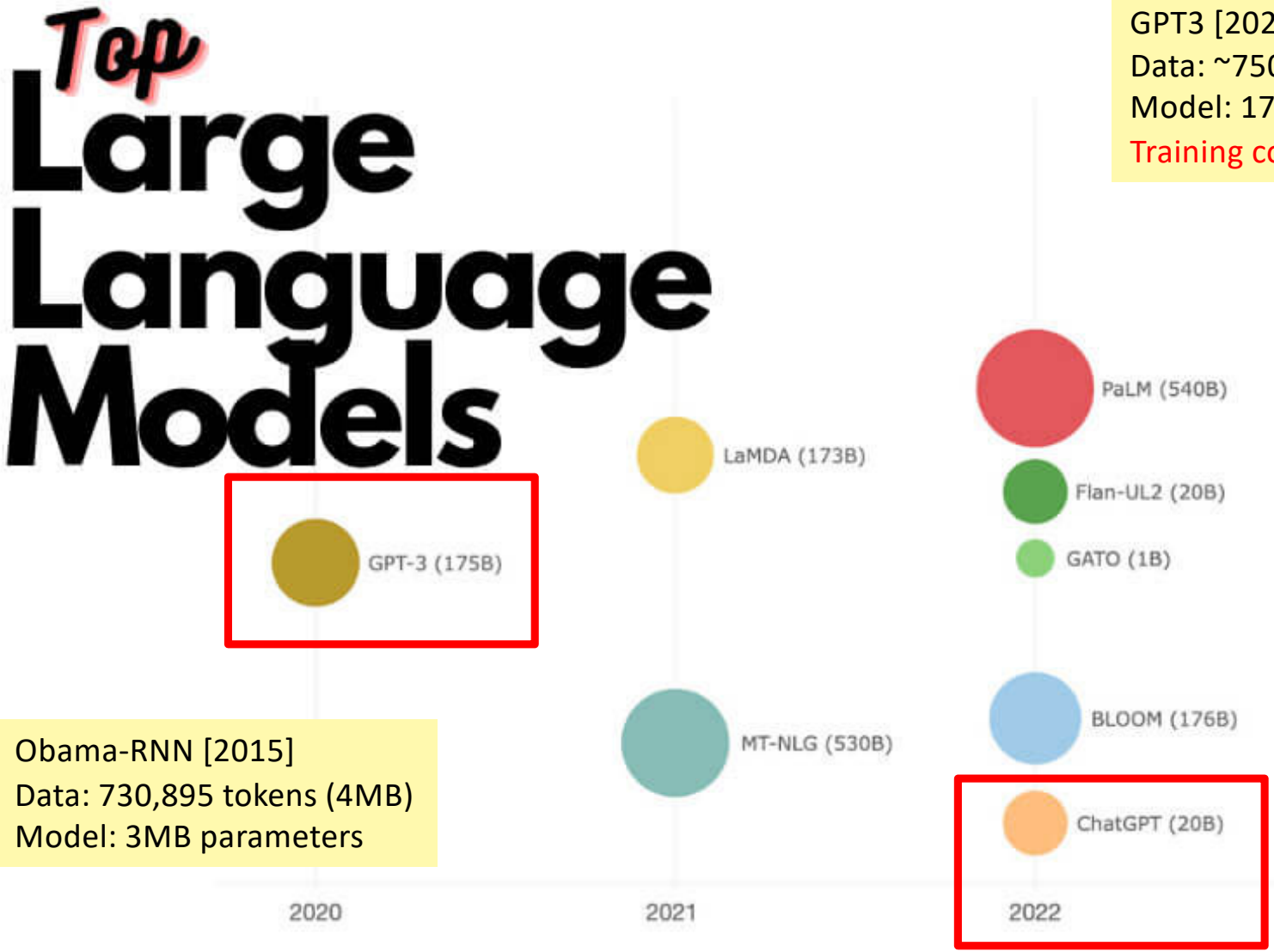
You have been using LM ☺ - Autocomplete

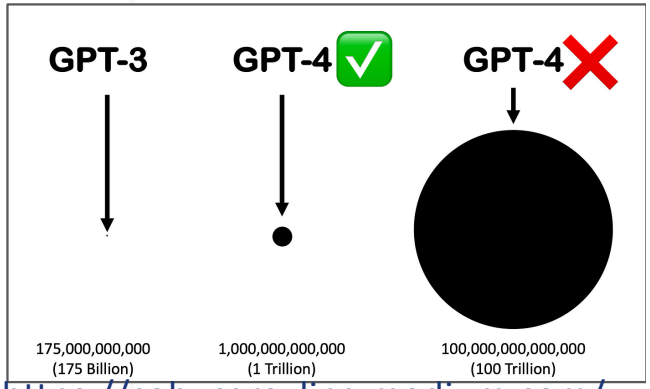




Top Large Language Models

GPT3 [2020]
Data: ~750GB (30,000x)
Model: 175B parameters (700,000x)
Training cost: $5M, equivalent to ~300 years





https://cobusgreyling.medium.com/what-are-realistic-gpt-4-size-expectations-73f00c39b832

Obama-RNN [2015]
Data: 730,895 tokens (4MB)
Model: 3MB parameters

https://vectara.com/top-large-language-models-llms-gpt-4-llama-gato-bloom-and-when-to-choose-one-over-the-other/

vectara

Step 1

Collect demonstration data and train a supervised policy.

LM

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.



Step 2

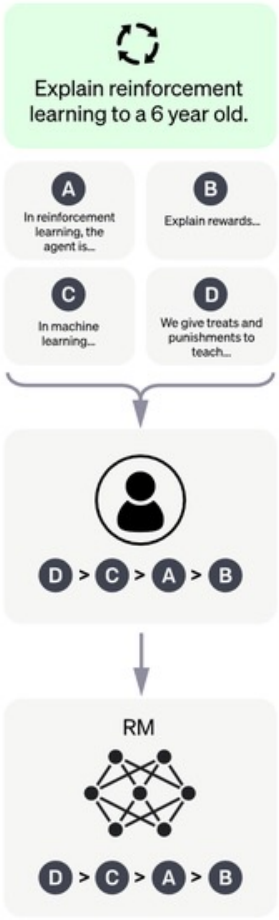
Collect comparison data and train a reward model.

Reward Model (Scoring Model)

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

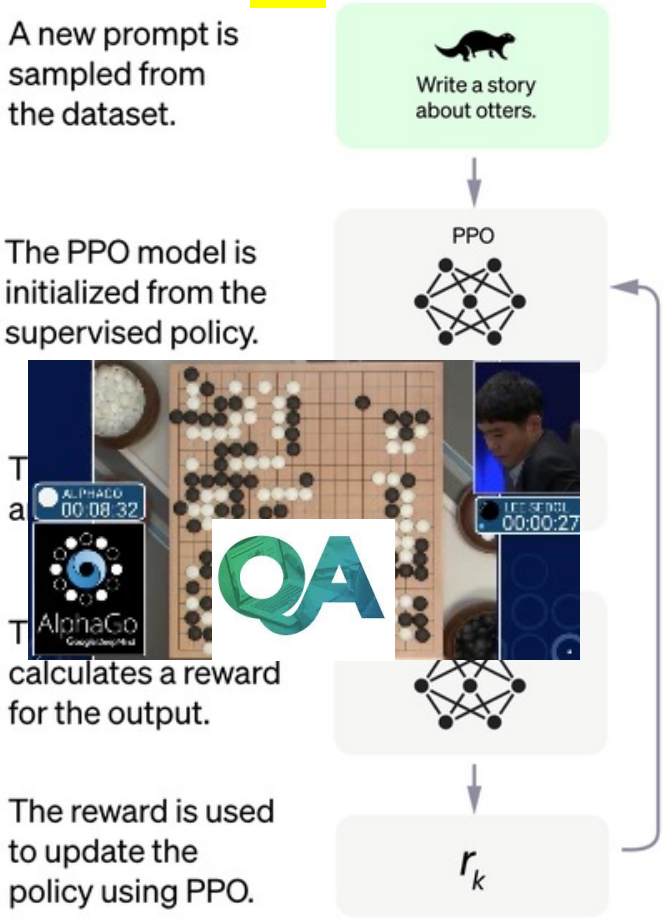
RL

A new prompt is sampled from the dataset.

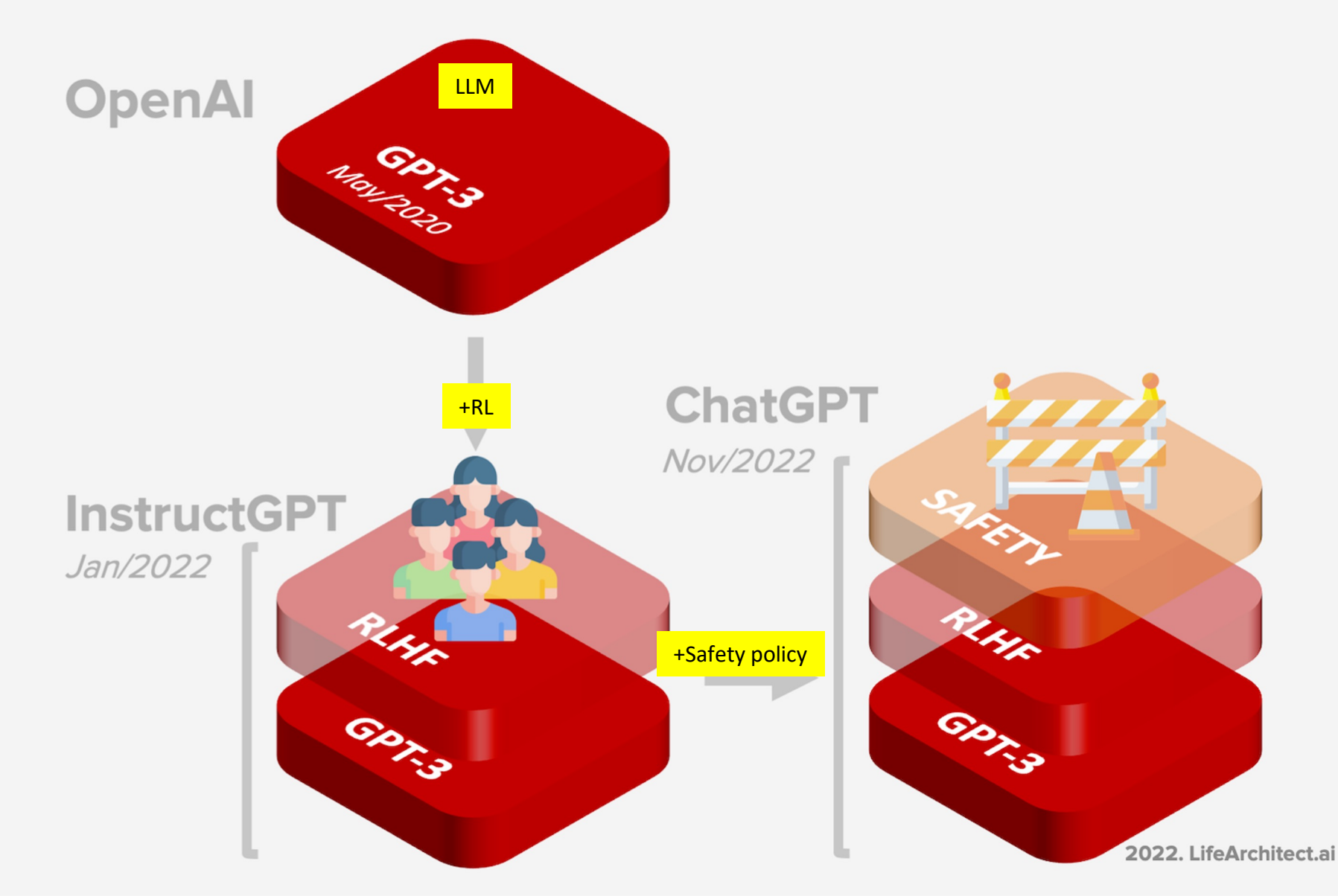
The PPO model is initialized from the supervised policy.

T
a
T
calculates a reward for the output.

The reward is used to update the policy using PPO.



https://www.hpc-ai.tech/blog/colossal-ai-chatgpt


OpenAI
LLM
GPT-3
May/2020
+RL
InstructGPT
Jan/2022
RLHF
GPT-3
+Safety policy
ChatGPT
Nov/2022
SAFETY
RLHF
GPT-3
2022. LifeArchitect.ai

Large + LM = LLM

"Large" is no longer just about parameter count.

Model	Year	Parameters	Key advancement
GPT-1	2018	117M	Transformer language modeling
GPT-2	2019	1.5B	Stronger zero-shot generation
GPT-3	2020	175B	In-context & few-shot learning
GPT-4	2023	ND (~1.7TB)	Multimodality + stronger reasoning
GPT-5	2025	ND (~300B)	Integrated reasoning + tool use
GPT-6 Astra	2026	Not disclosed (ND)	Computer use + end-to-end agentic work

Modern LLM capability ≠ parameter count alone

Pretraining + Architecture + Post-training + Inference-time Reasoning + Tools

Note: OpenAI has not publicly disclosed parameter counts for GPT-4, GPT-5, or GPT-6.

Evolution of LLMs

From language models to agentic models



GPT-1/2 Language Generation

GPT-3 In-Context Learning

GPT-4 Multimodality

GPT-5 Reasoning + Tools

GPT-6 Agentic Work

GPT-6 Astra (2026)

Reasoning • Computer Use • Tools • 1.05M Context

Source: OpenAI official model documentation, Sep 2026

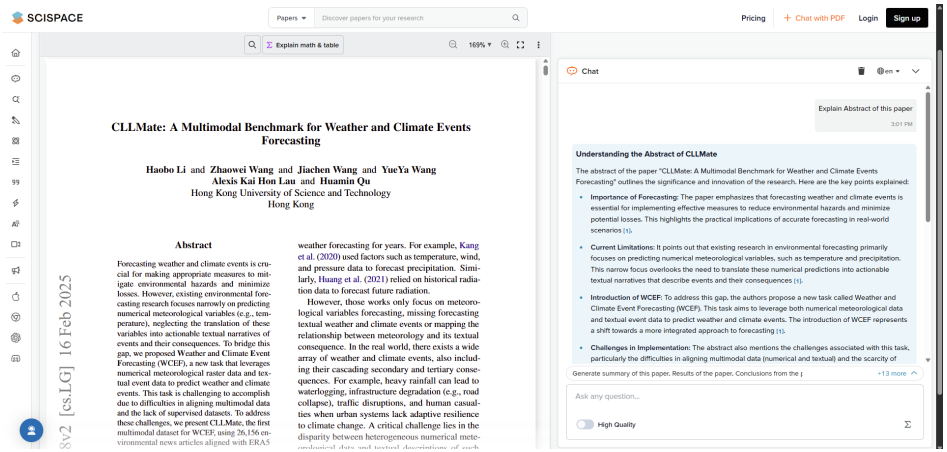
LLM Capabilities

ChatGPT

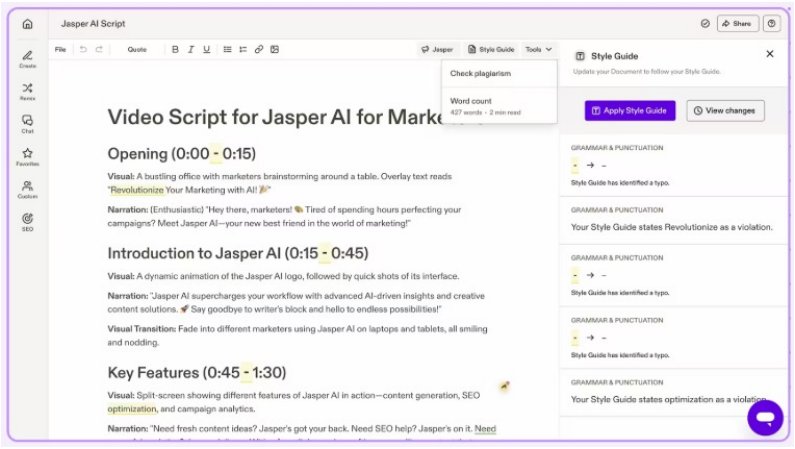
1) Assistant Chatbot



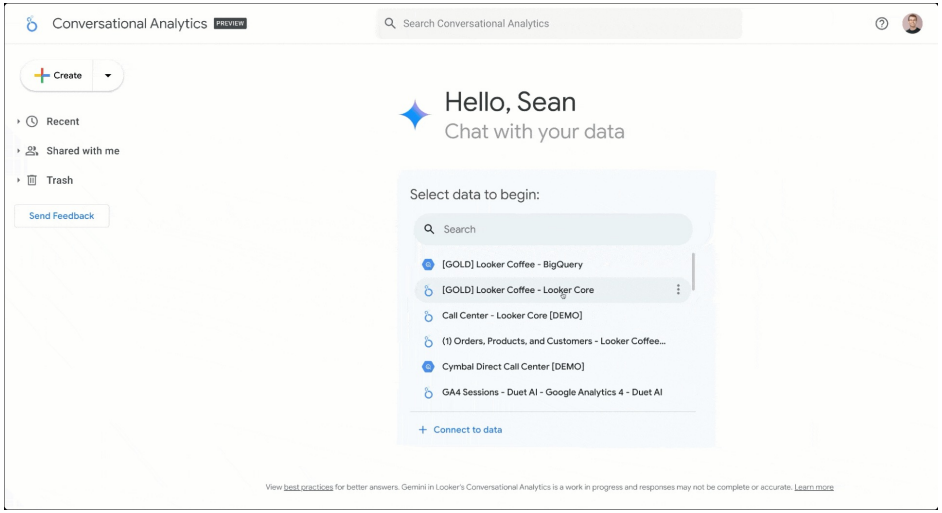
2) Text Summarization



3) Content Creation



4) Data Analytics



Modern LLMs in 2026 – Beyond Text Generation

2026 Trend: From specialized LLMs → unified multimodal, reasoning, and agentic models
Text → Multimodal → Reasoning → Tools → Agents

Category	Capabilities	Examples
1) Frontier / General-Purpose Models	Multimodal + reasoning + tools	GPT-6 Astra, Gemini 3.5, Claude 4.6
2) Reasoning Models	Complex reasoning, math, science, planning	GPT-6, Gemini 3.5, DeepSeek-R1/V-series
3) Multimodal / Omni Models	Text, image, audio, video understanding & generation	Gemini Omni, GPT-6, Gemini 3.5
4) Coding Models	Code generation, debugging, software engineering	GPT-6/Codex, Gemini 3.5, Claude 4.6
5) Agentic Models	Tool use, planning, computer/browser use, long-horizon tasks	GPT-6 Astra, Gemini 3.5, Claude 4.6
6) Long-Context Models	~1M-token context, long documents/codebases	GPT-6 Astra, Claude 4.6, Gemini family
7) Voice / Real-Time Models	Real-time speech, transcription, conversational voice	Gemini 3.5 Transcribe, real-time voice models

List of Interesting LLM models
(as of 25 Sep 2026)

1. OpenAI: GPT (28 Oct 2026)
2. Anthropic: Claude
3. Google: Gemini

4. Alibaba: Qwen
5. ByteDance: DeepSeek

1) OpenAI: GPT Models

- **GPT-6 Astra:** Highest capability for complex reasoning and coding; highest cost.
- **GPT-6.1 Sol:** Near-Astra performance at a lower cost; suitable for data analysis and everyday coding.
- **GPT-6.1 Terra:** Fast and efficient; strong in visual reasoning, computer use, and writing.
- **GPT-6 Luna:** Fastest and most affordable; ideal for high-volume tasks such as text classification and information extraction.

Aspect	GPT-6 Astra 🚀	GPT-6.1 Sol	(New!) GPT-6.1 Terra	GPT-6 Luna
Positioning	Most capable flagship	Near-flagship; lower cost	Fast, efficient & balanced	**Fastest & lowest-cost**
Best for	Hard reasoning · Coding · Research	Complex work · Coding · Agents	Visual reasoning · Computer use · Writing	**High-volume · Cost-sensitive**
Reasoning	low → max	low → max	none → max	**none → max**
Context	1.05M	1.05M	1.05M	**1.05M**
Max output	128K	128K	128K	**128K**
API Price (Input / Output per MTok)	$10 / $50	$2 / $10	$1 / $5	**$0.10 / $0.50**
Launch	Sep 3, 2026	Sep 29, 2026	Oct 26, 2026	**Sep 22, 2026**

2) Anthropic: Claude Models

- Claude Fable 5.1: Advanced reasoning and long-horizon agentic work; highest cost and slower responses.
- Claude Opus 5: Strong in complex agentic coding and enterprise tasks; moderate speed.
- Claude Sonnet 5: Balanced speed and intelligence; suitable for everyday tasks.
- Claude Haiku 4.5: Fastest and most affordable; ideal for high-volume tasks.

New!

	Claude Fable 5.1	Claude Opus 5	Claude Sonnet 5	Claude Haiku 4.5
	For demanding reasoning and long-horizon agentic work	For complex agentic coding and enterprise work	The best combination of speed and intelligence	The fastest model with near-frontier intelligence
Comparative latency	Slower	Moderate	Fast	Fastest
Pricing	$10 / input MTok $50 / output MTok	$5 / input MTok $25 / output MTok	$2 / input MTok $10 / output MTok	$1 / input MTok $5 / output MTok
Claude API ID	`claude-fable-5-1`	`claude-opus-5`	`claude-sonnet-5`	`claude-haiku-4-5-20251001`
Capabilities				
Thinking	Adaptive (always on)	Adaptive	Adaptive	Extended
Default effort	`high`	`high`	`high`	Not supported
Context window	1M tokens	1M tokens	1M tokens	200K tokens
Max output	128K tokens	128K tokens	128K tokens	64K tokens
Reliable knowledge cutoff	Jun 2026	May 2026	Jan 2026	Feb 2025

3) Google: Gemini Models

- Gemini 3.1 Pro: Advanced reasoning and creative work; highest cost.
- Gemini 3.8 Flash: Balanced speed and capability; suitable for coding, agents, and knowledge work.
- Gemini 3.5 Flash-Lite: Fast and affordable; ideal for high-volume tasks.
- Gemini 3.8 Live: Real-time voice interaction; ideal for voice agents and live dialogue.

	Gemini 3.1 Pro	(New!) Gemini 3.8 Flash	Gemini 3.5 Flash-Lite	(New!) Gemini 3.8 Live
Positioning	Complex tasks / advanced reasoning	**Main workhorse / latest Flash**	Fast & cost-efficient	Real-time voice
Best for	Complex reasoning · creative work	**Coding · agents · knowledge work**	High-volume · latency-sensitive tasks	**Voice agents · live dialogue**
Input	Text · Image · Video · Audio · PDF	Text · Image · Video · Audio · PDF	Text · Image · Video · Audio · PDF	Text · Image · Video · **Audio**
Context	1M	**1M**	1M	131K
Max output	64K	**64K**	64K	65K
Price / 1M tokens	**$2 input / $12 output** ≤200K prompt**$4 / $18** >200K	**$0.75 input / $3.75 output***	**$0.30 input / $2.50 output**	Text: **$0.75 / $4.50**Audio: **$3 / $12**
Launch	Feb 19, 2026	**Sep 2, 2026**	Jul 21, 2026	**Sep 15, 2026**

4) Alibaba: Qwen (general-purpose models)

Model	Main role	Launch Date
Qwen3.8-2.4T-A95B [Biggest]	**Large reasoning** and agent-oriented checkpoint	August 3, 2026
Qwen3.8-27B	**General-purpose multimodal model;** text, coding, reasoning, agents, and vision.	August 14, 2026
Qwen3.8-Flash-Next [Smallest – A6B] 125B main params + 51B N-gram emb	**Experimental architecture** focused on **efficiency**	August 26, 2026

Open releases: 2.4T-A95B on 12 Aug; 27B on 14 Aug 2026. Flash-Next is an architectural preview.

Qwen — official release repository

4) Alibaba: Qwen Specialized Models

Family	Model	Launch	Key capability	Reference
🎙️ **ASR**	**Qwen3-ASR 1.7B / 0.6B** + ForcedAligner	**28 Jan 2026**	52 languages/dialects · Streaming · · timestamps (**Thai ✓**)	Official Blog · GitHub
🔊 **TTS**	**Qwen3-TTS 1.7B / 0.6B**	**21 Jan 2026**	Voice Design · Voice Clone · expressive speech (No Thai Support)	Official Blog · GitHub
👁️ **Vision-Language**	**Qwen3-VL (2B · 4B · 8B · 32B · 30B-A3B (MoE) · 235B-A22B (MoE))**	**22 Sep 2025**	Image/video reasoning · OCR · visual agents (**Thai ✓**)	Official Blog · GitHub
🔎 **Embedding / Reranker**	**Qwen3-VL-Embedding / Reranker (2B, 8B)**	**7 Jan 2026**	Text + image + video → multimodal retrieval (**Thai ✓**)	Official Blog · GitHub

5) DeepSeek Models

Family	Models	Focus	Support Thai ?
General / Flagship	**DeepSeek-V4.1-Flash — 552B MoE** *(8B input / 16B output active)* ***[New!]***	General · Agents · Coding · Multimodal	✓ Usable* (DeepSeek does not explicitly list Thai as an officially supported language.)
Reasoning	**DeepSeek-R1 — 671B / 37B active**	Reasoning · Math · Coding	
Vision-Language	**DeepSeek-VL2 — 3.4B / 16.1B / 27.5B MoE**	Image + Text · OCR · Document / Chart understanding	⚠ Not verified
Coding	**DeepSeek-Coder-V2 — 16B / 236B**	Code generation · Code reasoning	⚠ Not verified
Math / Formal Reasoning	**DeepSeekMath-V2 — 685B · Prover-V2 — 7B / 671B**	Math · Proof · Theorem reasoning	⚠ Not verified

Comparison
(as of 25 Sep 2026)

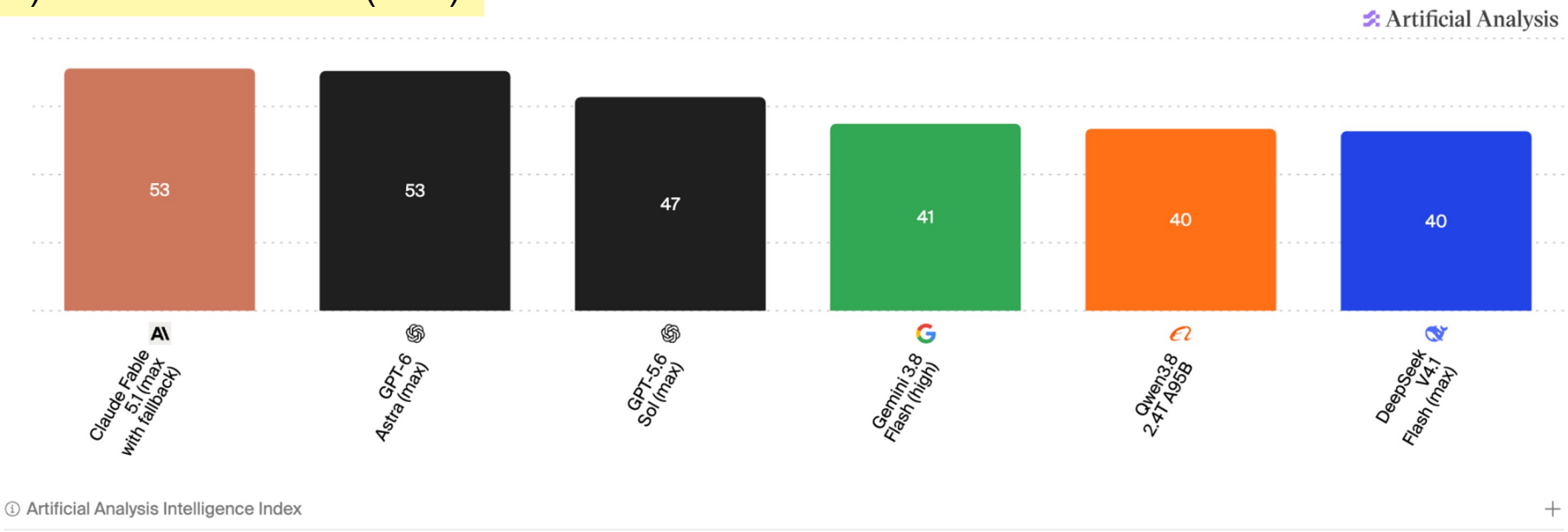
Artificial Analysis Intelligence Index

Artificial Analysis Intelligence Index v4.3 incorporates 10 evaluations: AA-Briefcase, GDPval-AA v2, AutomationBench-AA, Terminal-Bench 4.0, SciCode, Humanity's Last Exam, GDP.pdf, CritPt, AA-Omniscience, AA-LCR v1.1

6 of 651 models

Add model from specific provider

1) Human Last Exam (HLE)

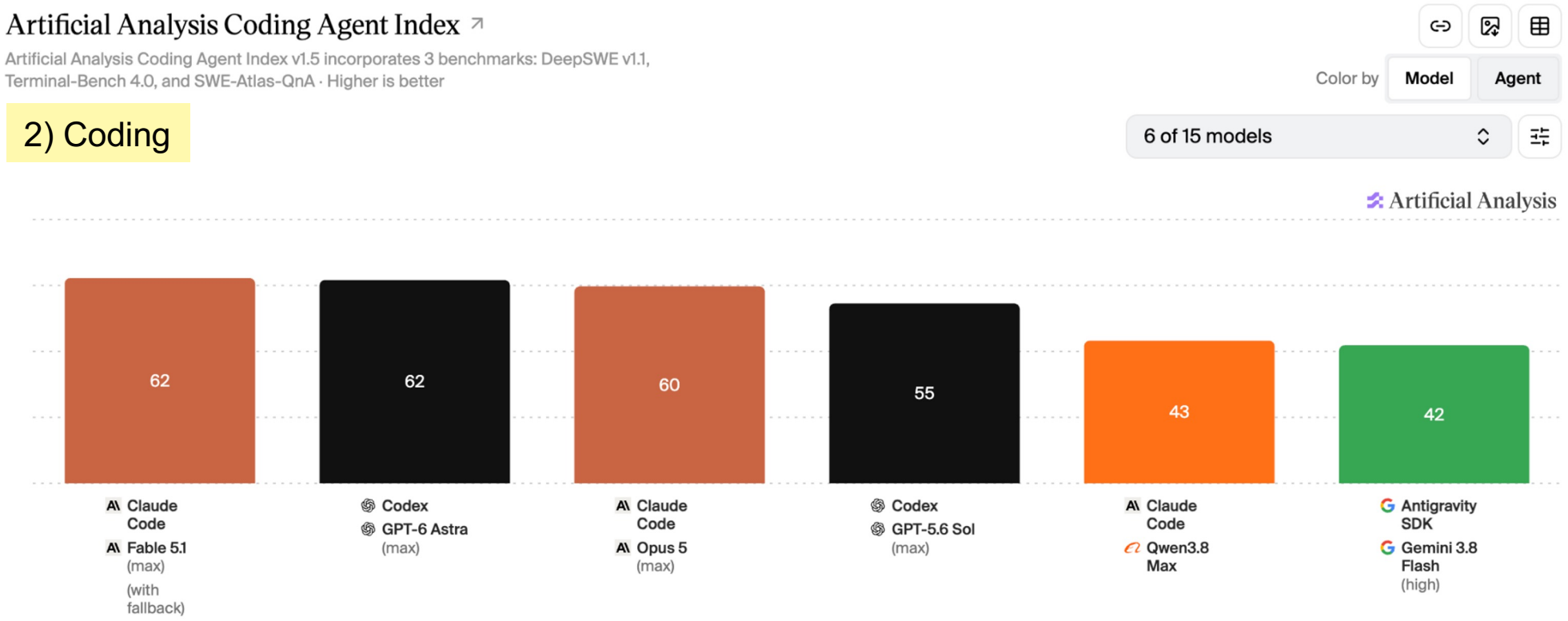


Intelligence

- High: Fable 5.1 , GPT-6 comparable
- Medium: Gemini 3.8 Flash , DeepSeek V4.1 (Open-source)

https://artificialanalysis.ai/

Useful Link: https://artificialanalysis.ai/



2) Coding

Coding

- High: Fable 5.1, GPT-6 Astra, Opus 5
- Medium: , GPT5.6-Sol
- Low: Gemini 3.8 Flash , Qwen 3.8 (Open-source)

3) Average Cost Per Task

Useful Link: https://artificialanalysis.ai/



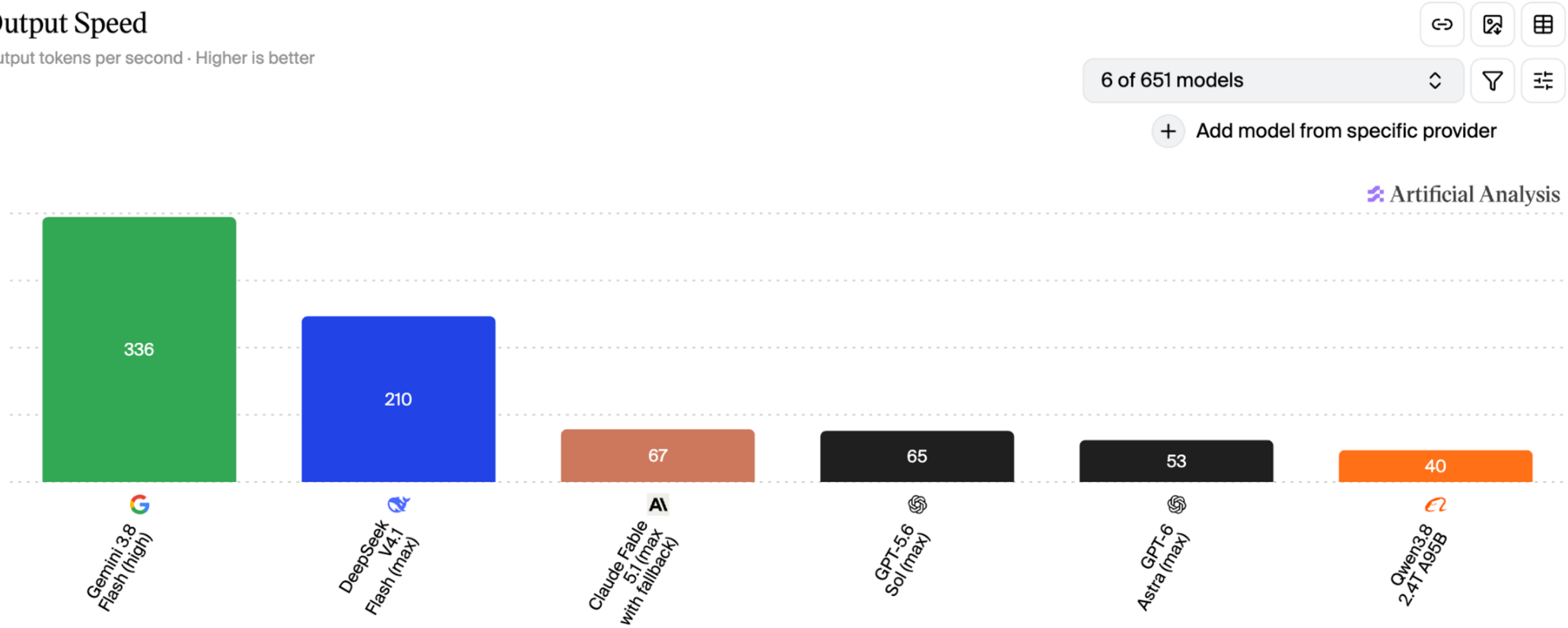
Approximate Cost per Task

- Fable 5.1 costs about 2× as much as GPT-6.
- GPT-6 costs about 1.5× as much as GPT-5.6.
- GPT-5.6 costs about 1.8× as much as Gemini 3.8 Flash.
- DeepSeek is very inexpensive.

4) Speed

Output Speed

Output tokens per second · Higher is better



2) Prompt Engineering

LLMs Need Instructions

Poor prompt

"Analyze this."

Better prompt

Analyze the customer reviews below.
Identify the three most common complaints and return a table with category, frequency, and one representative example.

Input, Output, Output Format

Clear task + constraints + output format → more predictable behavior

Prompt Engineering Techniques



servicenow

https://www.servicenow.com/ai/what-is-prompt-engineering.html

1) Prompt types

1) User prompt: the specific instructions or questions a user provides to an AI system to elicit a desired response.

2) System prompt: the foundational instructions that dictate an AI's behavior.

```
const response = await openai.chat.completions.create({
  model: "gpt-4o",
  messages: [
    {
      "role": "developer",
      "content": [
        {
          "type": "text",
          "text": `
            You are a helpful assistant that answers programming
            questions in the style of a southern belle from the
            southeast United States.
          `
        }
      ]
    },
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "Are semicolons optional in JavaScript?"
        }
      ]
    }
  ],
  store: true,
});
```

```
import anthropic

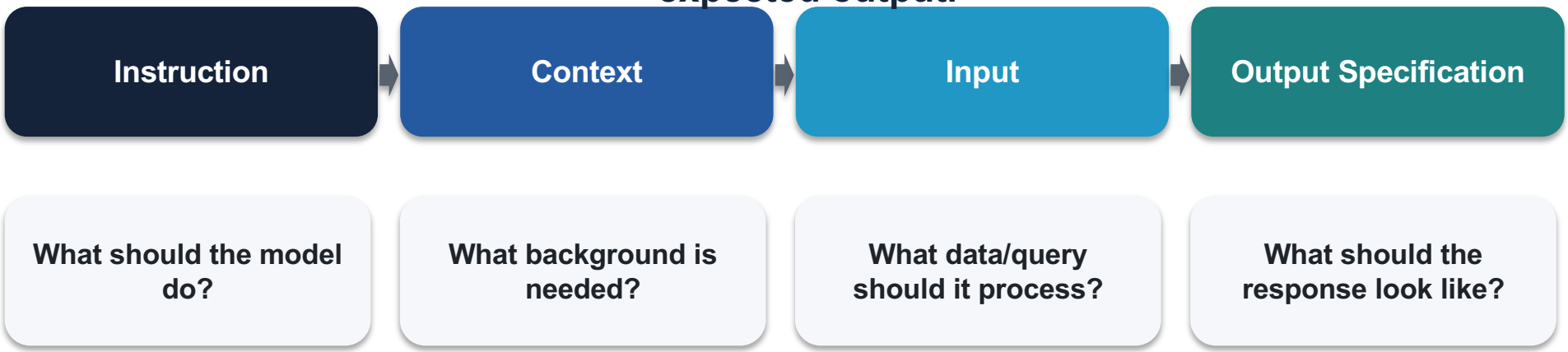
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=2048,
    system="You are a seasoned data scientist at a Fortune 500 company.", # <-- rol
    messages=[
        {"role": "user", "content": "Analyze this dataset for anomalies: <dataset>{
    ]
)

print(response.content)
```

2) Anatomy of a Prompt (ICIO)

A good prompt reduces ambiguity about both the task and the expected output.



Instruction: Extract the fruit names from the sentence.

Context: You are helping a grocery store create a shopping list.

Input: “I want to buy apples, bananas, and milk.”

Output Specification: Return only the fruit names as a bullet list.

Output:
- Apples
- Bananas

References: AnthPrompt

3) Zero-Shot vs. Few-Shot Prompting

Zero-shot

Classify this review as
Positive, Neutral, or Negative.

“Good food but expensive.”

Few-shot

“Excellent food.” → Positive
“It was okay.” → Neutral
“Very slow service.” → Negative

“Good food but expensive.” → ?

Examples communicate desired behavior, format, and decision boundaries.

References: AnthPrompt

4) Structured Outputs

Natural-language output

“The review is mostly positive, although the customer mentions price concerns.”

Structured output / schema

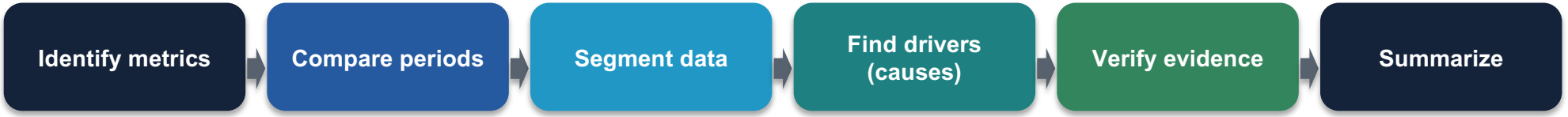
```
{
  "sentiment": "positive",
  "confidence": 0.87,
  "reason": "Good food; price concern"
}
```

Machine-readable outputs are essential for APIs, pipelines, and agents.

References: AnthPrompt

5) Decomposing Complex Tasks (Clear Instructions)

Question: “Why did sales decrease last month?”

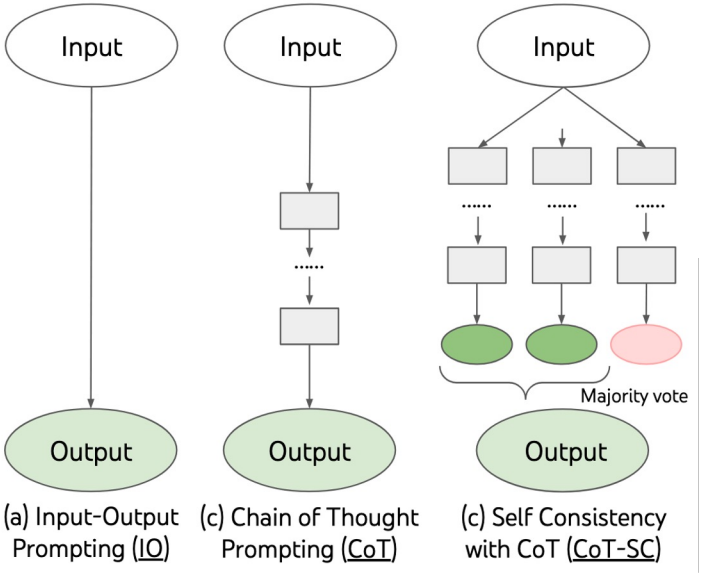


Structure the task when the problem naturally contains multiple steps.

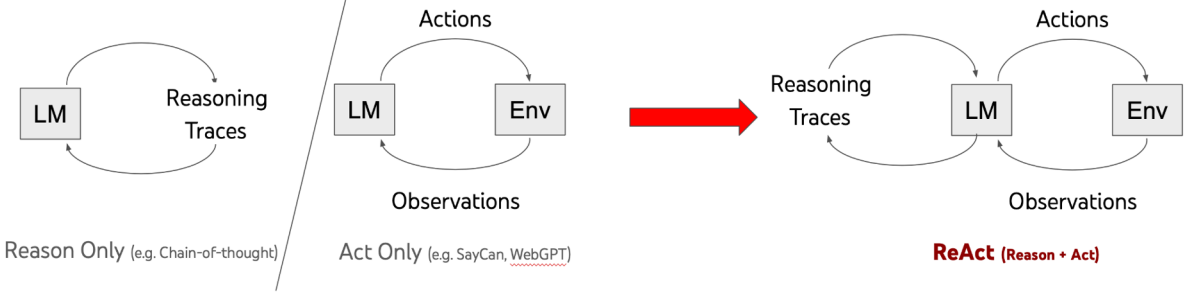
References: AnthPrompt

6) There are many more prompting techniques.

- 6.1) Chain-of-Thoughts (CoT)
- 6.2) Self-Consistency



- 6.3) ReAct (TBC):
  - Reason → Act → Observe → Repeat
  - Until reaching goals



3) Retrieval Augmented Generation (RAG)

Perfect prompt ≠ access to missing knowledge

Hallucination issue → LLM cannot access a private data!

ChatGPT

Who is Peerapon Vateekul?

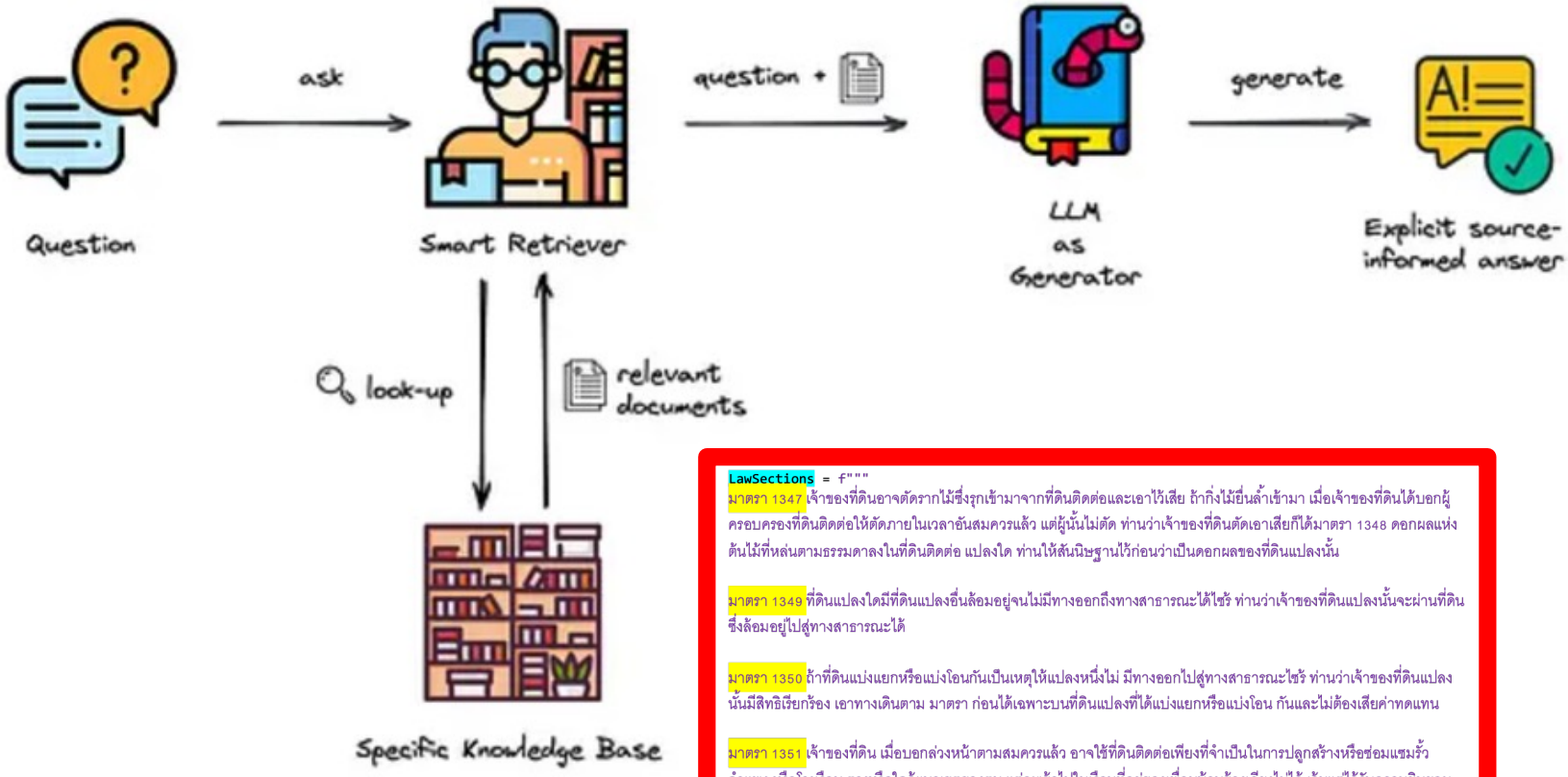
Peerapon Vateekul is a well-known journalist in Thailand, particularly in the mass media industry. He is known as a news anchor on Channel 3 and as the host of several popular television programs, including special news programs and documentaries broadcast on Channel 3 in Thailand.

Who is Narongdech Keeratiworanun?

Narongdech Keeratiworanun is a well-known Thai politician and businessman, and a member of a leading political party in Thailand. He also holds executive positions in various businesses.



Retrieval Augmented Generation (RAG)



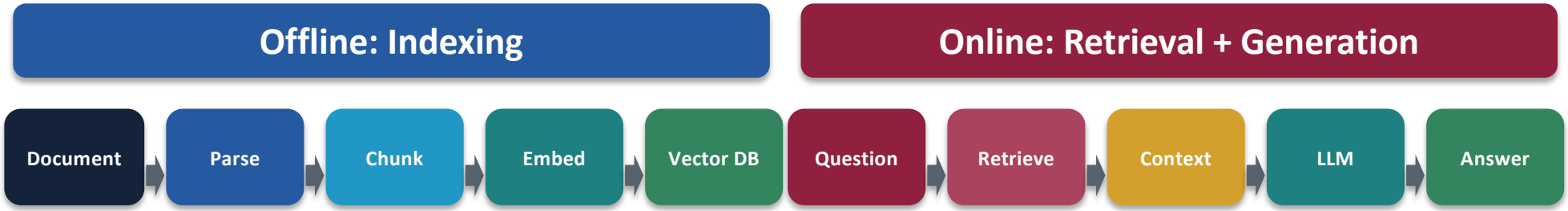

```
LawSections = f"""
มาตรา 1347 เจ้าของที่ดินอาจตัดรากไม้ซึ่งรุกเข้ามาจากที่ดินติดต่อและเอาไว้เสีย ถ้ากิ่งไม้ยื่นล้ำเข้ามา เมื่อเจ้าของที่ดินได้บอกผู้
ครอบครองที่ดินติดต่อให้ตัดภายในเวลาอันสมควรแล้ว แต่ผู้นั้นไม่ตัด ท่านว่าเจ้าของที่ดินตัดเอาเสียก็ได้มาตรา 1348 ดอกผลแห่ง
ต้นไม้ที่หล่นตามธรรมดาลงในที่ดินติดต่อ แปลงใด ท่านให้สันนิษฐานไว้ก่อนว่าเป็นดอกผลของที่ดินแปลงนั้น

มาตรา 1349 ที่ดินแปลงใดมีที่ดินแปลงอื่นล้อมอยู่จนไม่มีทางออกถึงทางสาธารณะได้ไซร้ ท่านว่าเจ้าของที่ดินแปลงนั้นจะผ่านที่ดิน
ซึ่งล้อมอยู่ไปสู่ทางสาธารณะได้

มาตรา 1350 ถ้าที่ดินแบ่งแยกหรือแบ่งโอนกันเป็นเหตุให้แปลงหนึ่งไม่ มีทางออกไปสู่ทางสาธารณะไซร้ ท่านว่าเจ้าของที่ดินแปลง
นั้นมีสิทธิเรียกร้อง เอาทางเดินตาม มาตรา ก่อนได้เฉพาะบนที่ดินแปลงที่ได้แบ่งแยกหรือแบ่งโอน กันและไม่ต้องเสียค่าทดแทน

มาตรา 1351 เจ้าของที่ดิน เมื่อบอกล่วงหน้าตามสมควรแล้ว อาจใช้ที่ดินติดต่อเพียงที่จำเป็นในการปลูกสร้างหรือซ่อมแซมรั้ว
กำแพง หรือโรงเรือน ตรงหรือใกล้แนวเขตของตน แต่จะเข้าไปในเรือนที่อยู่ของเพื่อนบ้านข้างเคียงไม่ได้ เว้นแต่ได้รับความยินยอม
"""
```

Basic RAG Architecture



RAG changes the context at inference time — it does not require changing model weights.

References: GoogleRAG

Step 1: Document Parsing & Chunking

DOCUMENT

Section 1
Paragraph...

Section 2
Paragraph...

Table...

Section 3...



Chunk 1
Section-aware

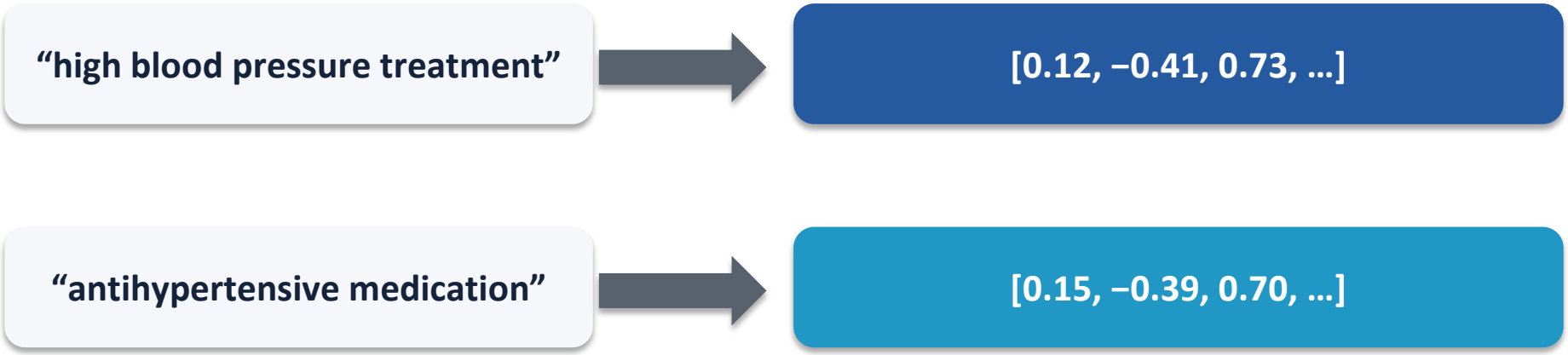
Chunk 2
Semantic

Chunk 3
Table-aware

- Too small → missing context
- Too large → noisy retrieval
- Prefer structure-aware splitting when possible

References: GoogleRAG

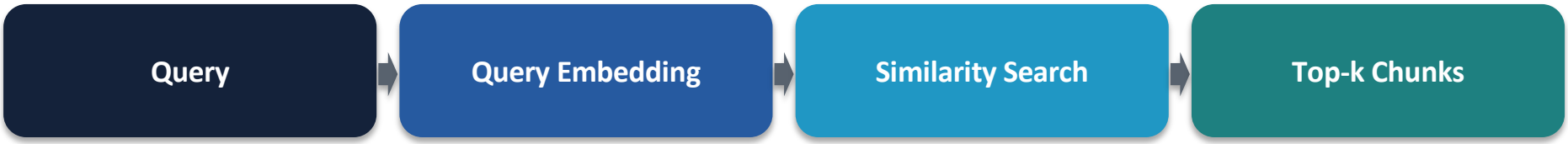
Step 2: Embeddings



Semantically similar text → nearby vectors in embedding space

References: GoogleRAG

Step 3: Vector Search



$$\cos(q,d) = (q \cdot d) / (||q|| \, ||d||)$$

Question: Is semantic similarity always sufficient?

References: GoogleRAG

Sparse vs. Dense Retrieval

Sparse / Keyword (e.g., BM25)

- Exact terms and rare keywords
- Names, IDs, codes
- Lexical matching

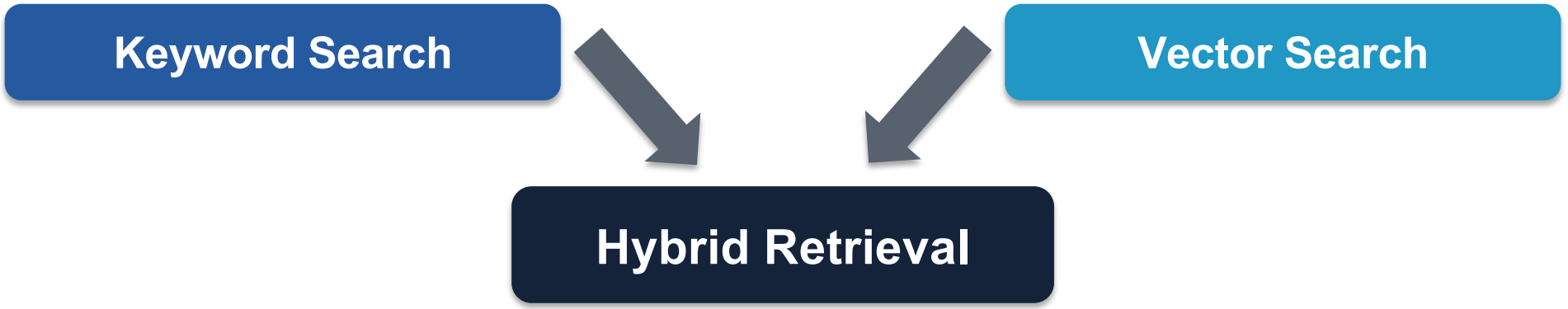
Dense / Vector

- Semantic similarity
- Paraphrases
- Conceptual matching

Example: “CU-2190513” may favor keyword search over semantic search.

References: GoogleRAG

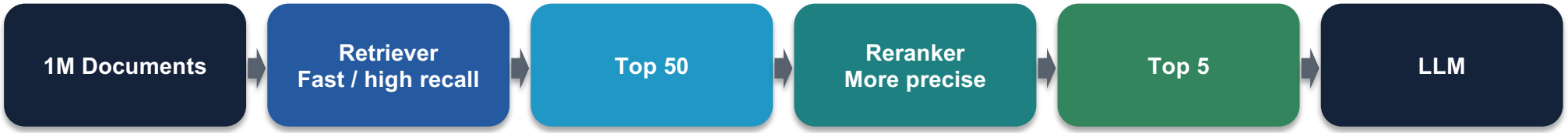
Hybrid Search



Semantic search does not replace keyword search — they complement each other.

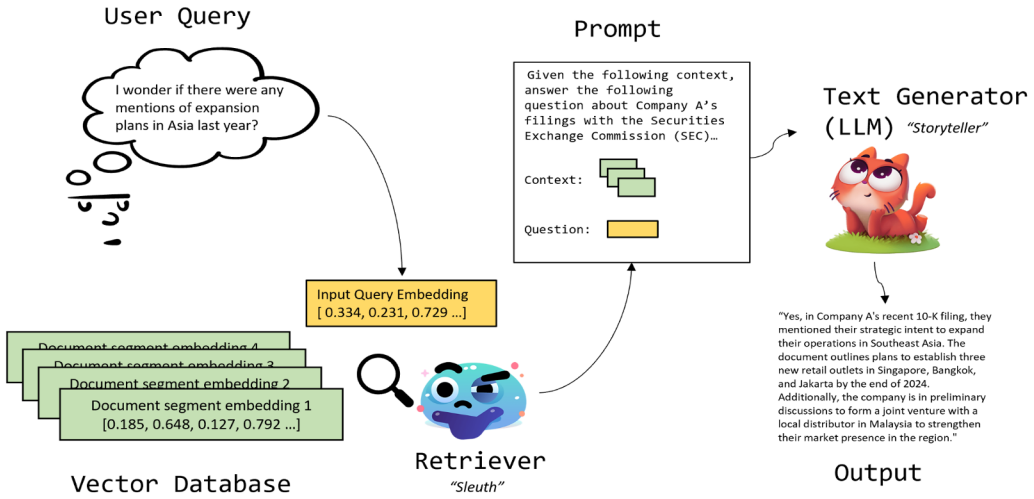
References: GoogleRAG

Retrieve → Rerank → Step 4: Generate Answers



Retrieve broadly

Rerank precisely

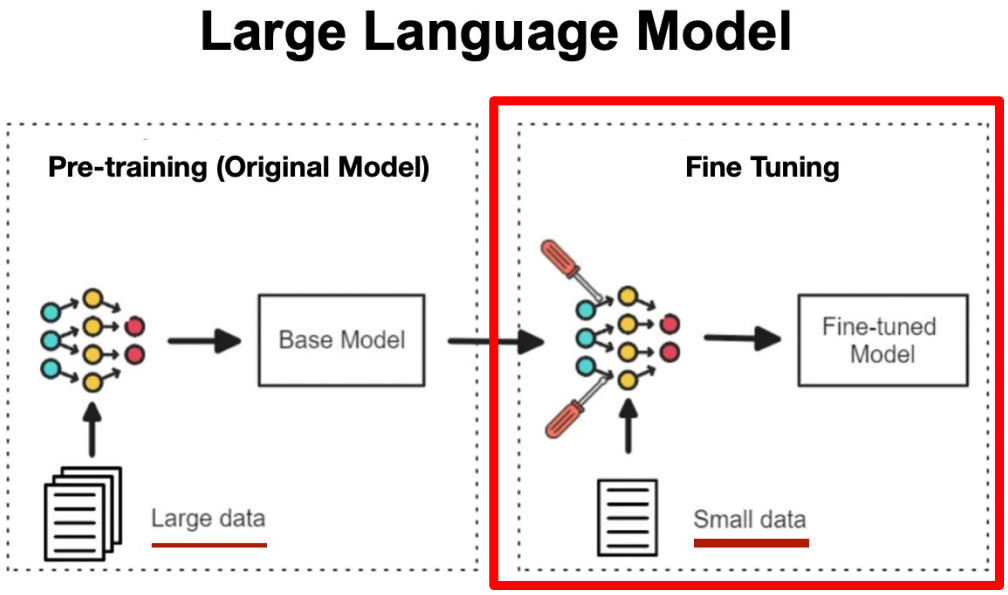


References: GoogleRAG

4) Fine-Tuning (briefly)

Fine-tuning

Fine-tuning is retraining a pre-trained LLM with your own domain-specific data. The goal is to make the model **behave** like an expert in a specific domain. It helps the model better understand **specialized** language, style, and tasks.



Prompting vs. RAG vs. Fine-tuning

- RAG improves what the model knows, while Fine-tuning improves how the model behaves. They can work together.



Method	What changes?	Example
Prompting	Instructions and examples in the prompt	Define review categories
RAG	External information retrieved into the context	Answer from a restaurant policy
Fine-tuning	Model parameters through additional training	Learn consistent review labeling (expensive)

Fine-Tuning Example and Evaluation

TASK: CLASSIFY SENTIMENT BY ASPECT

Input: “The food was delicious, but the service was slow.”

Expected output / Label: {"food": "positive", "service": "negative"}

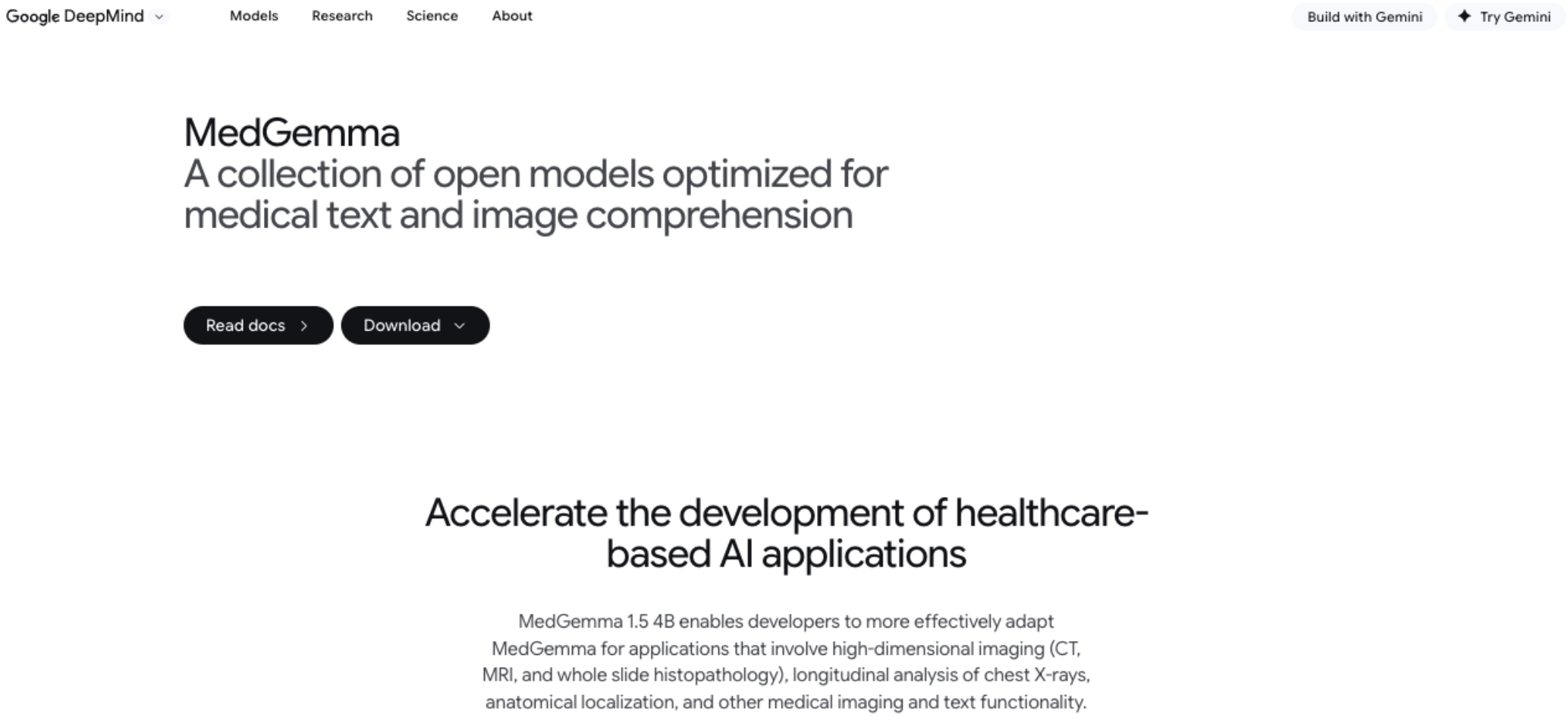
Training process

1. Prepare representative examples
2. Split training, validation, and test data
3. Fine-tune the pretrained model
4. Test against a few-shot baseline

What to measure

Quality: accuracy or F1
Format: valid output structure
Efficiency: time and cost

Use fine-tuning when measured gains justify the training and maintenance effort.



https://deepmind.google/models/gemma/medgemma/

FinBERT

FinBERT is a pre-trained NLP model to analyze sentiment of financial text. It is built by further training the BERT language model in the finance domain, using a large financial corpus and thereby fine-tuning it for financial sentiment classification. Financial PhraseBank by Malo et al. (2014) is used for fine-tuning. For more details, please see the paper FinBERT: Financial Sentiment Analysis with Pre-trained Language Models and our related blog post on Medium.

The model will give softmax outputs for three labels: positive, negative or neutral.

How to use

```
from transformers import pipeline

classifier = pipeline("text-classification", model="ProsusAI/finbert")
classifier("Stocks rallied and the British pound gained.")
# [{'label': 'positive', 'score': 0.898...}]
```

defog/**sqlcoder-7b-2** Like 452 Follow Defog.ai 234

Text Generation | Transformers | Safetensors | GGUF | llama | text-generation-inference | License: cc-by-sa-4.0

Model card | Files and versions xet | Community 51

Edit model card

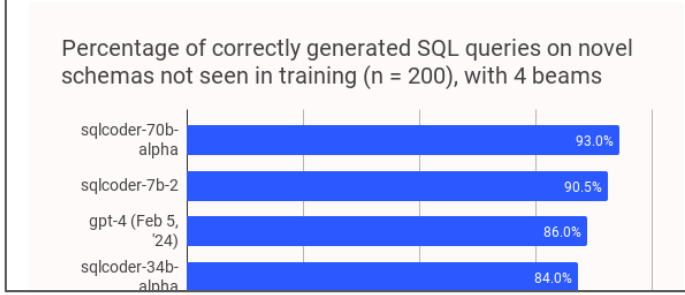
Update notice

The model weights were updated at 7 AM UTC on Feb 7, 2024. The new model weights lead to a much more performant model – particularly for joins.

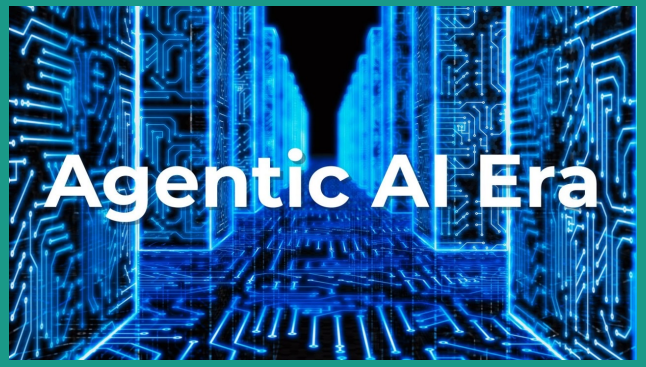
If you downloaded the model before that, please redownload the weights for best performance.

Model Card for SQLCoder-7B-2

A capable large language model for natural language to SQL generation.



5) Tool Use & Agentic AI



Codex

Google Antigravity

CLAUDE CODE

5.1 From Knowledge to Action

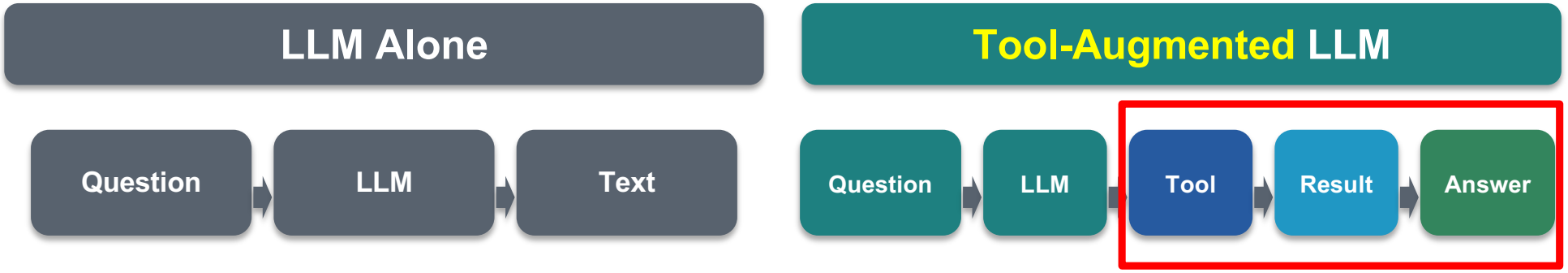


Run Python | Query SQL | Search Web | Call API | Send Email

What if solving the task requires multiple actions and decisions?

References: AnthAgents

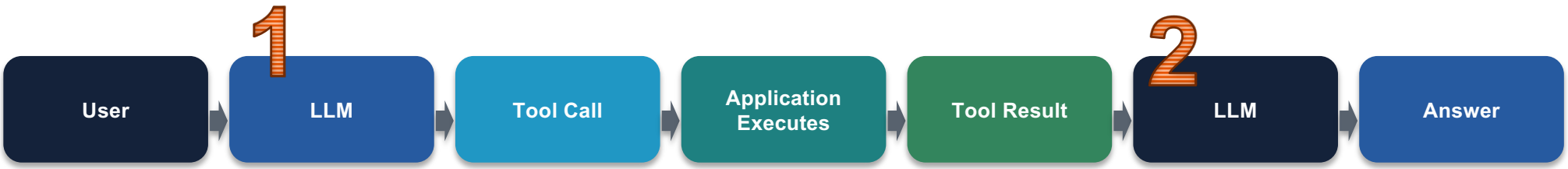
LLM + Tools



Tools extend what the model can access, compute, and change in the external world.

References: AnthAgents

Function / Tool Calling



User: "What was total revenue in Q2?"

1) LLM requests:
query_sales(quarter="Q2", metric="revenue")

3) Tool returns: 1,450,000 THB

4) LLM responds:
"Q2 revenue was 1.45M THB."

The model requests the tool call; the application/runtime executes it.

References: AnthAgents

Different Types of Tools

Retrieve	Compute	Data	Act
Web search · RAG · Files	Calculator · Python	SQL · Databases	Email · Calendar · APIs · Browser

RAG itself can be viewed as one information-retrieval capability available to an agent.

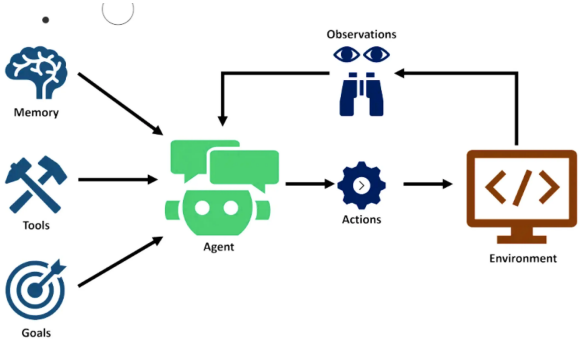
References: AnthAgents

5.2 What Is an AI Agent? Agent = Goal Oriented + Loop

Agent = Model + Instructions + Tools + Memory/State + Loop



The model (agent) dynamically chooses the next action based on observations.



Reasoning: ReAct = Reasoning + Action

Published as a conference paper at ICLR 2023

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao*,1, Jeffrey Zhao2, Dian Yu2, Nan Du2, Izhak Shafran2, Karthik Narasimhan1, Yuan Cao2

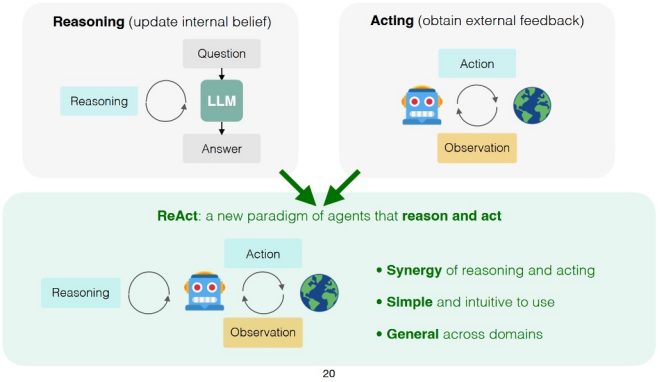
1Department of Computer Science, Princeton University
2Google Research, Brain team
1{shunyuy,karthikn}@princeton.edu
2{jeffreyzhao,dianyu,dunan,izhak,yuancao}@google.com

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named ReAct, to a diverse set of language and decision making tasks

cs.CL] 10 Mar 2023

Complex goals require a loop:
decide → act → observe → decide again.



ReAct: Reason + Act



Goal
Compare Q2 sales with last year.

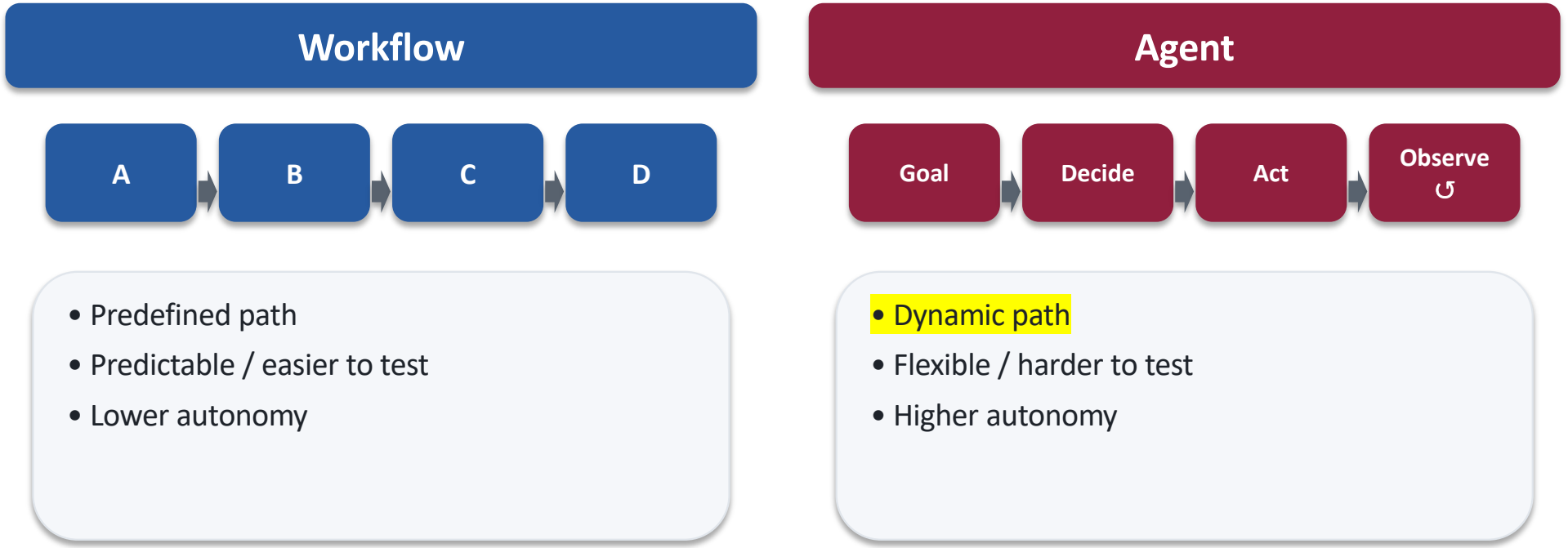
Action
query_database(Q2, 2026)

Observation
Revenue = 12.5M

ReAct interleaves reasoning/decision-making with actions and observations.

References: ReAct

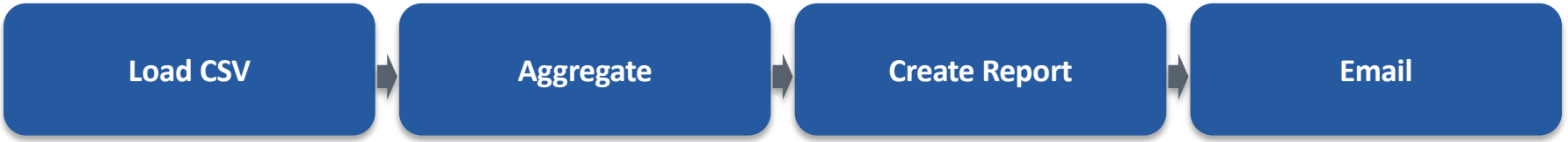
Workflow vs. Agent



References: AnthAgents

When NOT to Use an Agent

Every night: load CSV → aggregate → create report → email



Known steps? Use a workflow.

Use an agent when the path depends on changing intermediate results.

References: AnthAgents

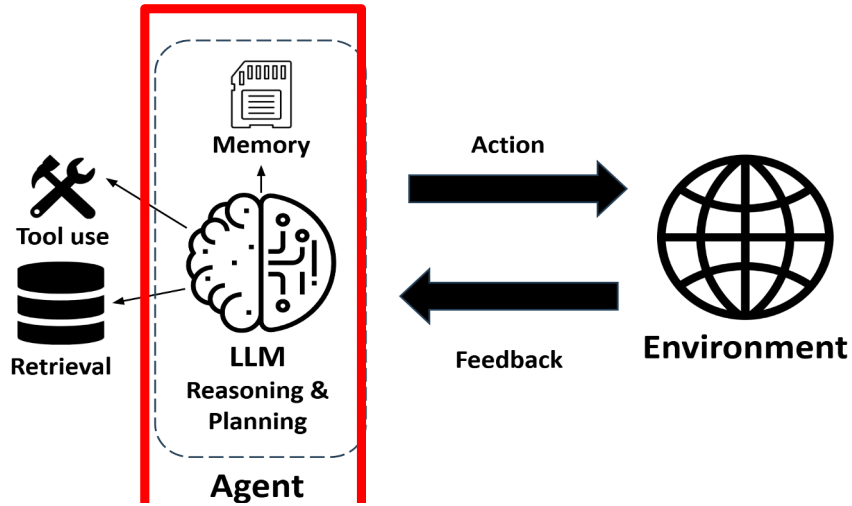
Single Agent vs. Multi-Agent

More agents ≠ better system: specialization can add cost, latency, and coordination errors.

Single Agent

Agent

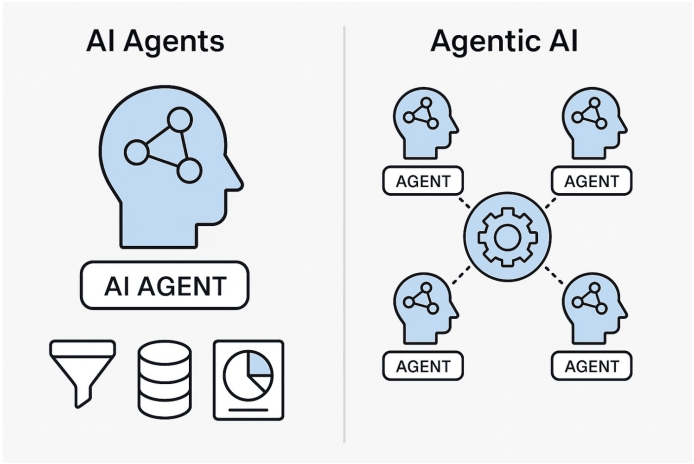
Search | Python | SQL | Email



Multi-Agent

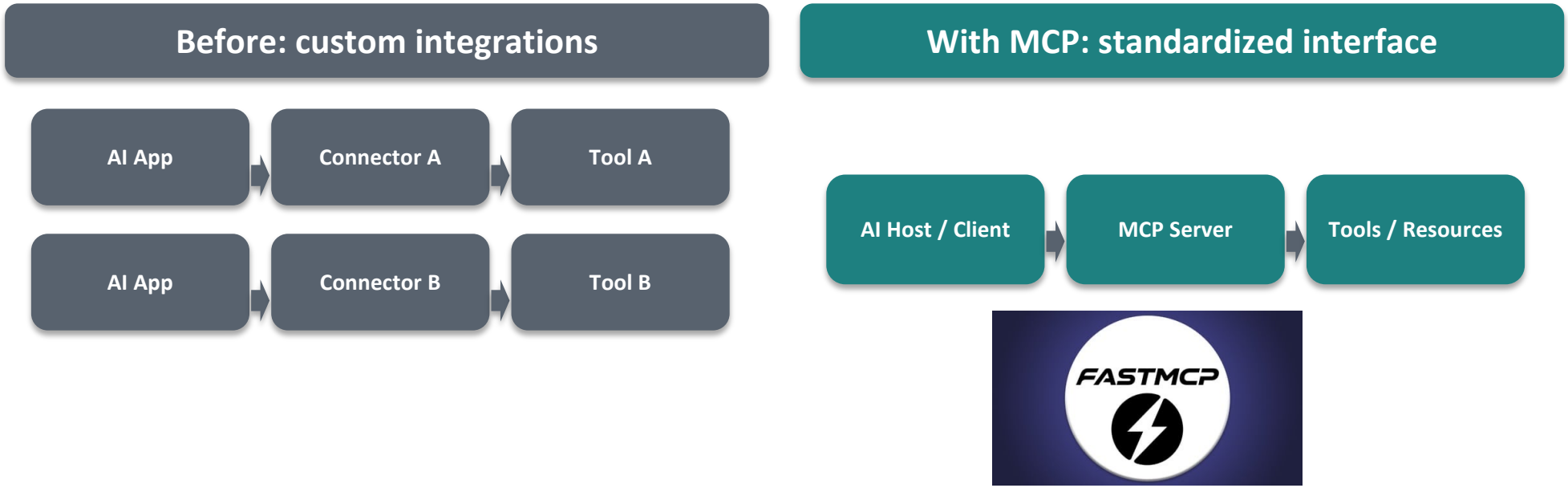
Supervisor

Researcher | Analyst | Writer



References: AnthAgents

Model Context Protocol (MCP): Connecting AI to Tools and Data



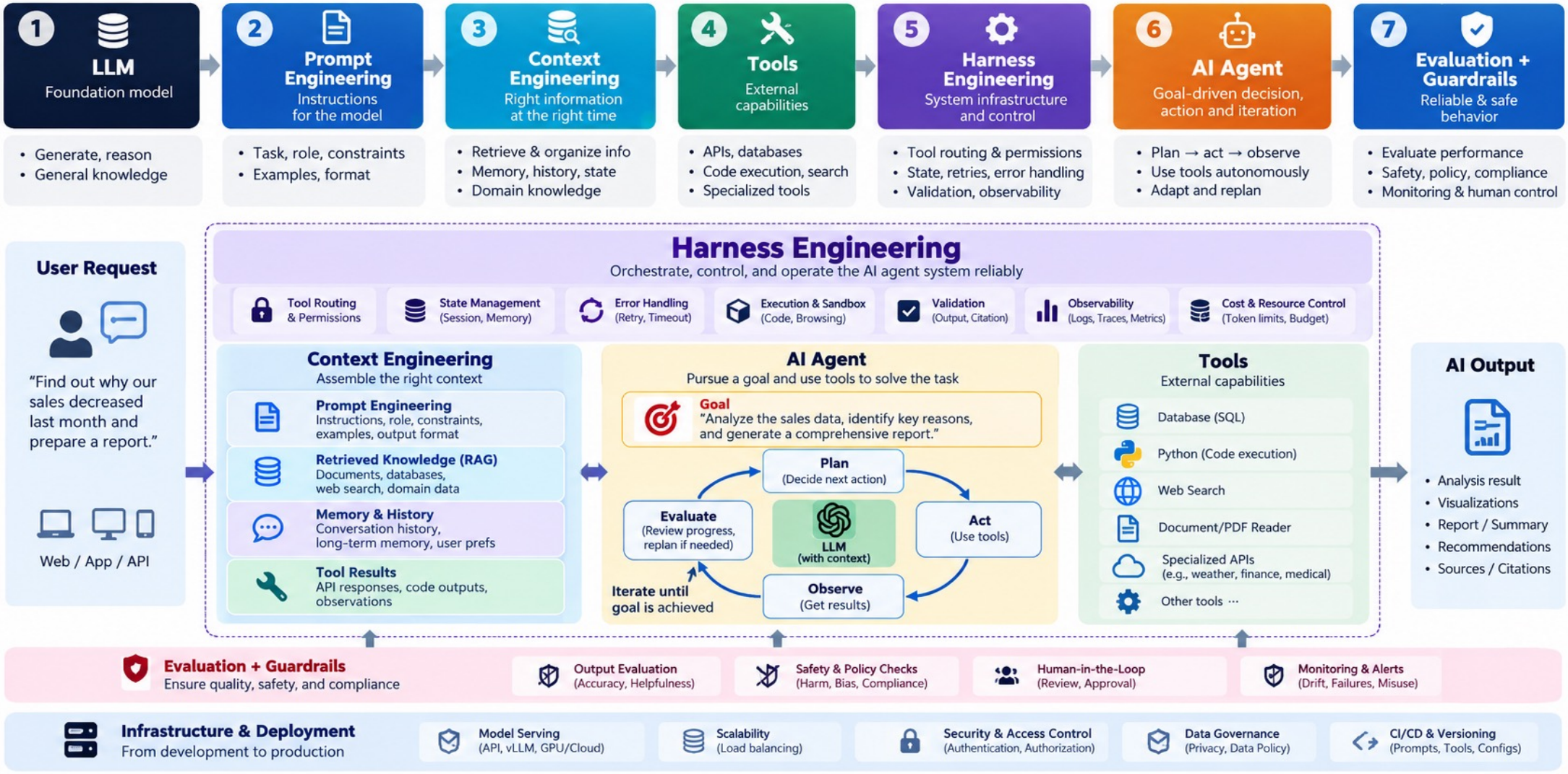
MCP standardizes how AI applications connect to external tools and context sources.

References: MCP

The Big Picture: Modern GenAI System

Beyond this class

From a foundation model to a reliable, real-world AI system



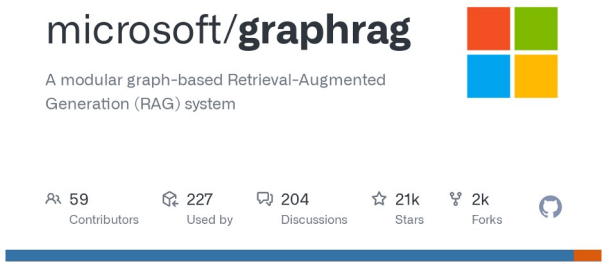
7) LLM Tools

LLM Tools

LLM Frameworks







Agentic Frameworks



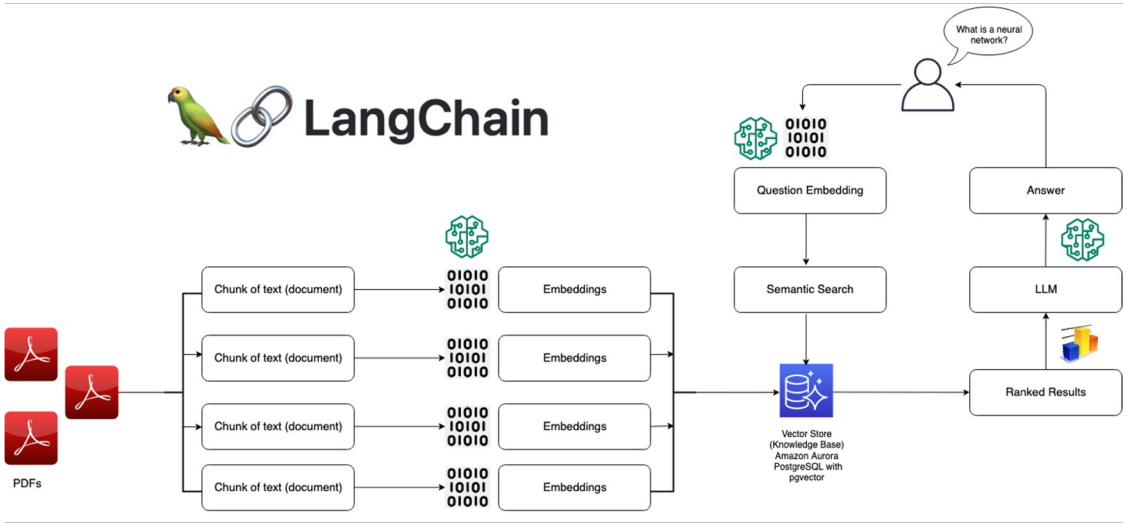






1) LangChain

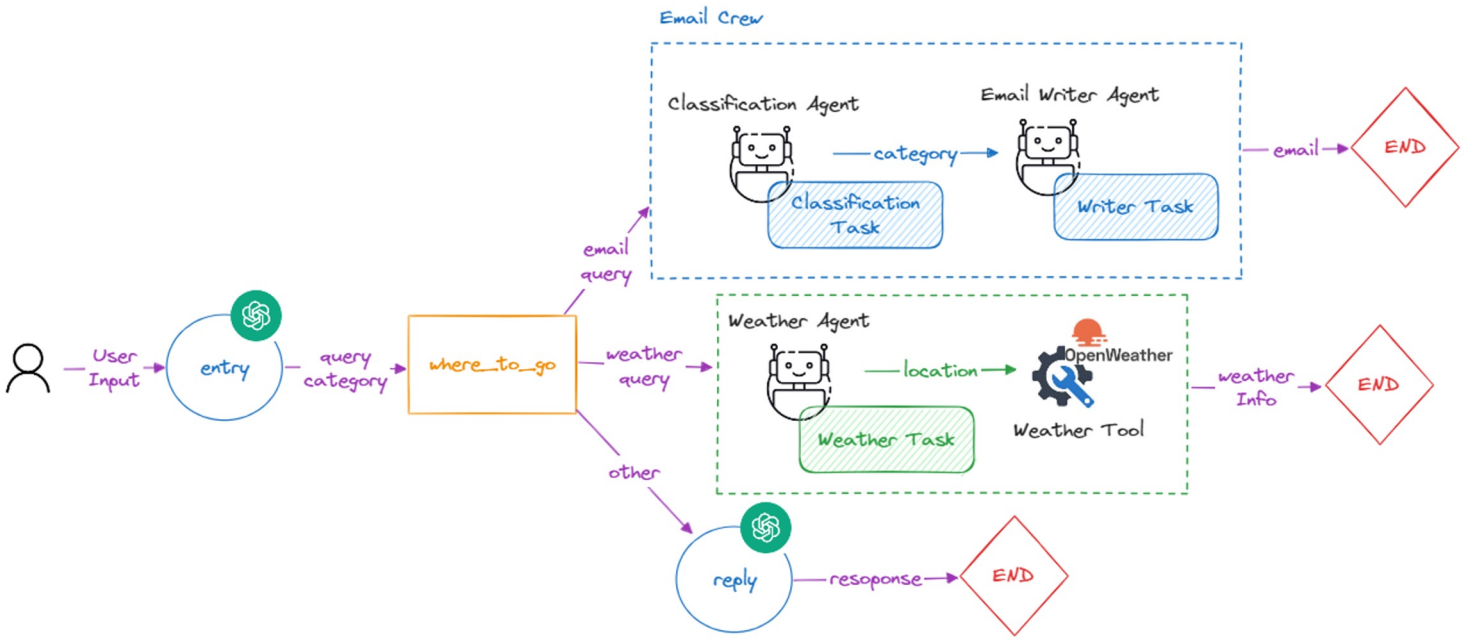
LangChain is a framework that enables developers to build powerful LLM-based applications by managing external data connections, orchestrating multi-step interaction logic, and enhancing model capabilities such as context handling and tool execution, making it ideal for creating AI agents, chatbots, RAG systems, and complex automation workflows.





2) LangGraph

LangGraph is an orchestration framework for complex agentic systems and is more low-level and controllable than LangChain agents.



3) LangSmith (LLMOps)

Langsmith is designed with AI model debugging and orchestration in mind.

1) Observability

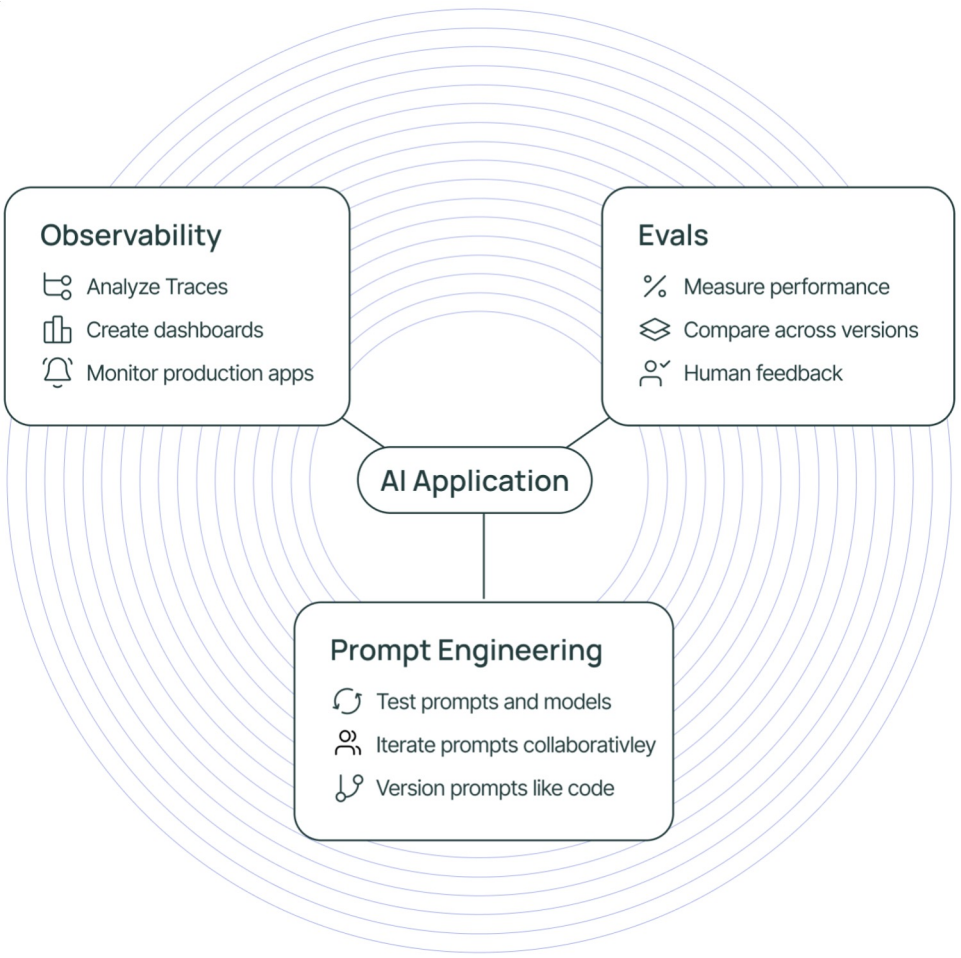
Analyze traces in LangSmith and configure metrics, dashboards, alerts based on these.

2) Evals

Evaluate your application over production traffic — score application performance and get human feedback on your data.

3) Prompt Engineering

Iterate on prompts, with automatic version control and collaboration features.



https://www.langchain.com/langgraph

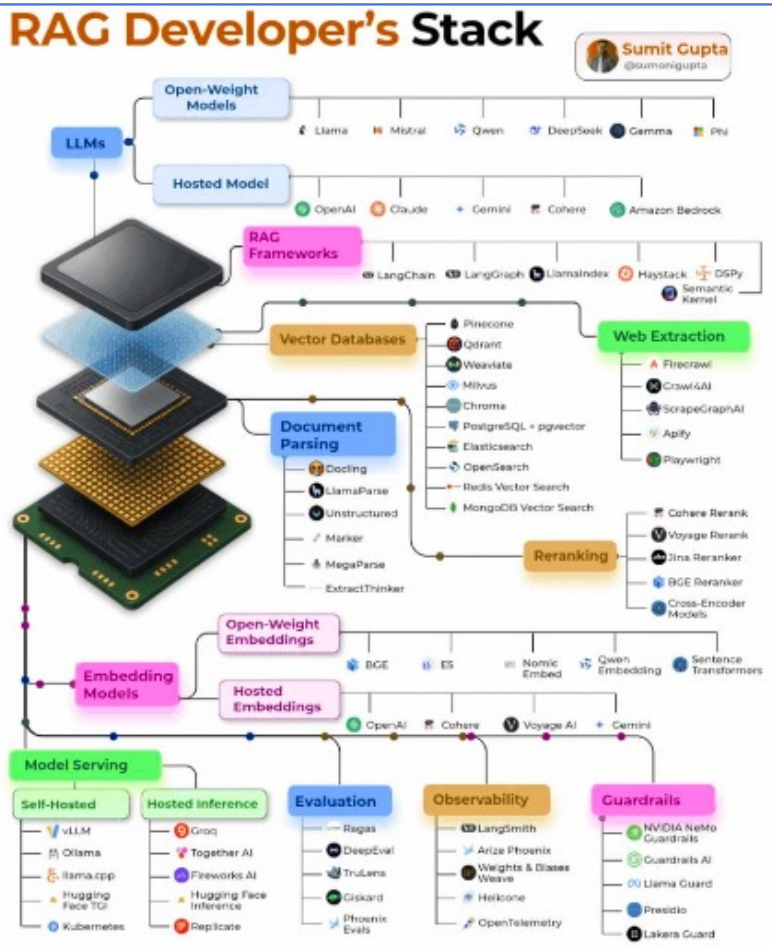
Credit Dr.Toe's slide (NECTEC)

The agentic RAG stack, layer by layer

Nine layers from infrastructure to governance. Pick one tool per layer and swap layers independently.



Inside one agentic RAG loop: route, retrieve, reason, check



LEVELS 0 TO 2
Deployment and infrastructure, evaluation and monitoring, foundation models

LEVELS 3 TO 5
Orchestration frameworks, vector databases, embedding models

LEVELS 6 TO 8
Data ingestion and extraction, memory and context management, safety and governance

RIGHT PANEL
The same stack grouped by job: parsing, re-ranking, serving, observability and guardrails

Please make sure to get all API keys before the lab.

1. gemini: https://aistudio.google.com/app/apikey
2. openai: https://platform.openai.com/account/api-keys
3. nvidia: https://build.nvidia.com/settings/api-keys

4. langsmith : smith.langchain.com
5. (Go to Developer (top right) → API Keys → Create API Key. Copy the key and save it securely.)

6. (optional) https://huggingface.co/settings/tokens (for 7_8 finetuning)

Q&A

Peerapon.v@chula.ac.th